ZERO TO HERO / STUDY NOTEBOOK

SPRING SECURITY

INTERVIEW HANDBOOK

144 ORIGINAL QUESTIONS

144 ANSWERED FOLLOW-UPS

21 CHAPTERS + 10 LABS



Java 17+ | Security 6.5 | Boot 3.5

Simple English • Visual Notes • Code + Tests

PREMIUM ILLUSTRATED STUDY EDITION

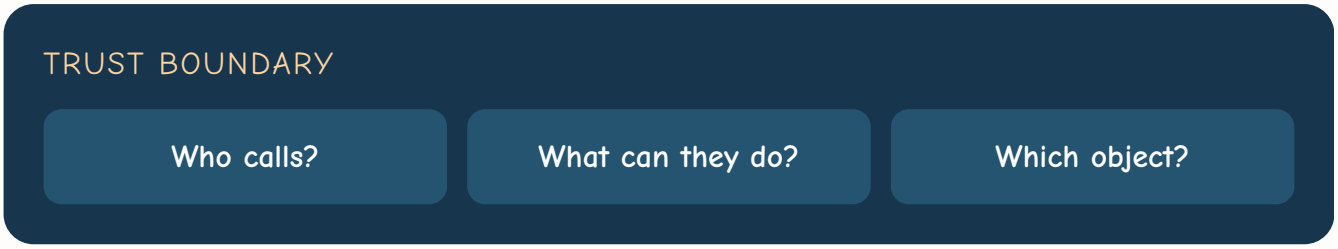
How to study these notes

This edition transforms the supplied Spring Security guide into a fast-to-read, hand-lettered learning notebook. Original interview questions and follow-up prompts are retained in the same order. The answers use short sentences, practical examples, and a security decision flow.

Each question page has five parts:

1. A direct interview answer
2. A security flow diagram
3. The original code or scenario
4. The most important pitfall
5. The follow-up question with a new answer

The security rule to remember:



Do not stop when a request returns **200**. Also prove that wrong users, wrong tenants, malformed credentials, and untrusted clients cannot perform forbidden actions.

Version: Java 17+, Spring Security 6.5 with Spring Boot 3.5. Most code blocks are focused excerpts from the source guide, not complete independent applications. Test them in a compatible Spring project before reuse.

BROWSE THE NOTEBOOK

21 topics · 144 questions

Chapter map

Follow the original chapter order from first authentication concepts to senior production incidents. Use the question IDs to find what you need.

01 Security foundations

02 Servlet filter architecture

03 Authentication model and context

04 Password authentication and user storage

05 Sessions, login, logout and remember-me

06 HTTP authorization and chain boundaries

07 Method and object-level security

08 CSRF and browser credential behavior

09 CORS, headers and transport protection

10 JWT structure and cryptographic trust

11 JWT decoding, validation and authority mapping

12 OAuth 2.0 flows and client responsibilities

13 OIDC login and federated identity

14 Resource-server tokens, refresh and revocation

15 Authorization servers and signing-key lifecycle

16 Multi-tenant and service-to-service boundaries

17 Reactive and asynchronous security

18 Security testing and negative cases

19 Operational security and abuse controls

20 Modern API choices and legacy migration

21 Senior troubleshooting and architecture scenarios

How Spring Security protects a request

SERVLET SECURITY

HTTP request → Selected chain → Business handler

1. Choose the chain. Spring Security selects the first matching SecurityFilterChain. A missing catch-all can expose unexpected routes.

2. Authenticate. Check the configured credential type: password, session, bearer JWT, or opaque token. A valid signature alone is not an authorization decision.

3. Authorize. Check method/endpoint permission and real object ownership. Tenant safety must also hold on cache hits and asynchronous paths.

4. Verify denial. Wrong issuer, audience, key, owner, role, and missing CSRF tokens must produce the intended response without a business side effect.

Three questions about every token

PROVE TRUST BEFORE ACCESS

Who issued it? | Who is it for? | What rights?

Issuer + signature + algorithm: Trust only configured issuers and known verification keys. The `kid` header is a key selection hint, not an arbitrary fetch URL.

Audience + time: A correctly signed token can still be for the wrong service, expired, or not yet valid.

Permissions + ownership: Scopes and roles authorize selected operations. They do not automatically authorize access to every customer's data.

JWT is not OAuth: JWT is a token representation. OAuth defines delegated access. OIDC adds a client identity layer. A signed JWT is usually readable, not encrypted.

Q001–Q006

01

Security foundations

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Credential evidence

Trusted identity

Right to access?

001 · What problem does Spring Security solve?

002 · How do authentication and authorization differ?

003 · How do principal, credentials and authorities differ?

004 · How do roles and authorities differ?

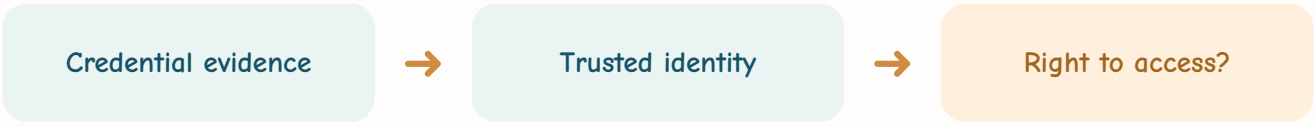
INTERVIEW QUESTION 001

What problem does Spring Security solve?

EXPLAIN IN ONE BREATH

It supplies authentication, authorization, and common security protections integrated with Spring applications.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Request -> security processing -> permitted handler
Denied request -> defined authentication or access response
```

Why it works: The framework supplies mechanisms, but the application must define trusted identities, permissions, credential handling and data ownership. A dependency alone cannot decide who owns an order.

⚠ **Interview trap:** Security configuration is not a complete threat model.

FOLLOW-UP QUESTION — ANSWERED

Which business rule cannot be inferred from a valid login?

A successful login does not prove that the user owns a particular order, invoice, or tenant record. Check ownership for each requested object.

INTERVIEW QUESTION 002

How do authentication and authorization differ?

EXPLAIN IN ONE BREATH

Authentication establishes identity; authorization decides whether that identity may perform a particular action.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
User Sai signs in successfully.
Sai requests another tenant's invoice.
Identity valid -> ownership check can still deny access.
```

Why it works: A successful login is not universal permission. Protect both operation-level capability and the target object's ownership. Keep denied outcomes distinct from malformed input or unavailable data.

⚠ **Interview trap:** A valid token does not authorize every object id.

FOLLOW-UP QUESTION — ANSWERED

How would you test cross-tenant denial?

Create tenants A and B, sign in as A, request B's record, expect denial and confirm the database record was not read or changed.

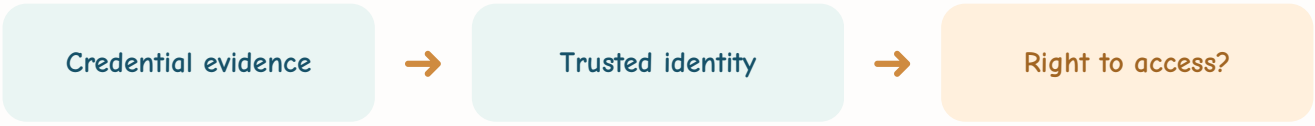
INTERVIEW QUESTION 003

How do principal, credentials and authorities differ?

EXPLAIN IN ONE BREATH

Principal identifies the caller, credentials are evidence used to authenticate, and authorities describe granted capabilities.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
Object principal = auth.getPrincipal();
Collection<? extends GrantedAuthority> rights = auth.getAuthorities();
```

Why it works: The principal can be UserDetails, Jwt, or another supported identity object. Credentials should be handled minimally and may be erased after authentication. Never assume a password remains available in an authenticated token.

⚠ **Interview trap:** Casting every principal to your entity is unsafe.

FOLLOW-UP QUESTION — ANSWERED

What principal type does JWT authentication normally expose?

With Spring's usual JWT resource-server setup, the principal is a Jwt, wrapped by JwtAuthenticationToken. Custom converters may change that shape.

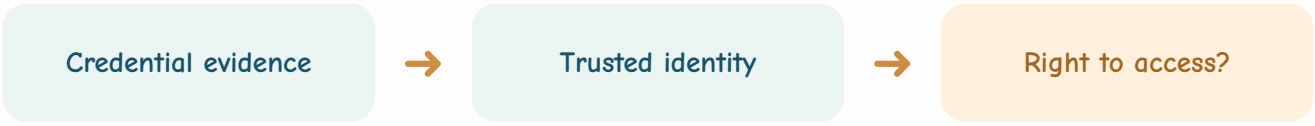
INTERVIEW QUESTION 004

How do roles and authorities differ?

EXPLAIN IN ONE BREATH

Authorities are permission strings; role helpers conventionally add a ROLE_ prefix when checking a role.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
hasAuthority("orders:approve") -> exact authority
hasRole("ADMIN") -> normally checks ROLE_ADMIN
// Role prefix can be configured in supported integrations.
```

Why it works: Use a consistent mapping policy. A token claim named roles is not automatically mapped into Spring's expected strings. Grouping fine-grained permissions into roles is an application decision.

⚠ **Interview trap:** hasRole("ROLE_ADMIN") commonly applies the wrong expectation.

FOLLOW-UP QUESTION — ANSWERED

How would you map identity-provider groups?

Map only trusted, allowlisted provider groups into approved GrantedAuthority values; do not accept an arbitrary roles claim from an untrusted issuer.

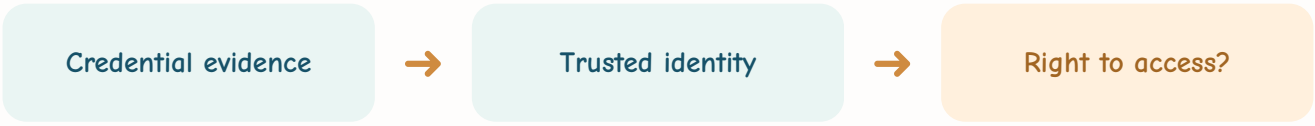
INTERVIEW QUESTION 005

What do 401, 403 and login redirects mean?

EXPLAIN IN ONE BREATH

401 requests or reports authentication failure in an API contract; 403 denotes refused access; browser login entry points can redirect instead.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Bearer API without valid credentials -> commonly 401
Authenticated caller lacking permission -> commonly 403
Form-login browser request -> commonly redirect to login
```

Why it works: The selected entry point, access-denied handler, and failure source determine the actual response. A CSRF failure can also produce 403. Do not diagnose solely from status without finding the denying component.

⚠ **Interview trap:** Anonymous access denial does not always produce the same response as an authenticated denial.

FOLLOW-UP QUESTION — ANSWERED

Which handler controls API unauthenticated responses?

AuthenticationEntryPoint controls unauthenticated API responses, normally 401 with an appropriate WWW-Authenticate challenge.

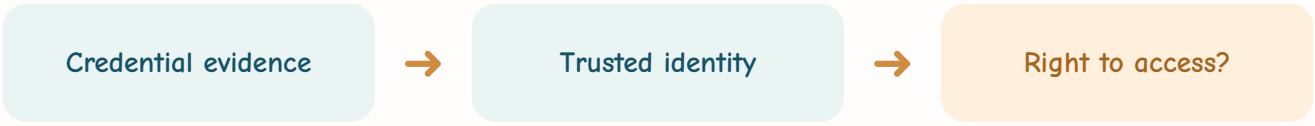
INTERVIEW QUESTION 006

Why does adding the security starter change a Boot app?

EXPLAIN IN ONE BREATH

Boot activates supported security defaults when its conditions match; explicit beans let the application own configuration.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
spring-boot-starter-security -> relevant security classes
No custom chain -> default Boot security behavior
Custom SecurityFilterChain -> application controls its rules
```

Why it works: For the baseline, default web authentication and generated credentials are development starting points. Separate user-details defaults can have different back-off conditions from the web chain.

⚠ **Interview trap:** A custom chain alone does not configure a production identity store.

FOLLOW-UP QUESTION — ANSWERED

Which bean replaces the default user service?

Define your own UserDetailsService (or other authentication setup) to replace Boot's generated in-memory user defaults.

02

Servlet filter architecture

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL



007 · How do DelegatingFilterProxy and FilterChainProxy differ?

008 · What is a SecurityFilterChain bean?

009 · How are multiple security chains selected?

010 · How do securityMatcher and requestMatchers differ?

INTERVIEW QUESTION 007

How do DelegatingFilterProxy and FilterChainProxy differ?

EXPLAIN IN ONE BREATH

DelegatingFilterProxy bridges the servlet container to a Spring bean; FilterChainProxy selects and executes a Spring Security filter chain.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Servlet container
-> DelegatingFilterProxy
-> FilterChainProxy
-> selected SecurityFilterChain -> DispatcherServlet
```

Why it works: The bridge allows Spring-managed security infrastructure to participate in container requests. FilterChainProxy owns chain selection and other security integration. These names refer to different responsibilities, not alternative configurations.

⚠ **Interview trap:** A controller may never run when a security filter rejects the request.

FOLLOW-UP QUESTION — ANSWERED

Where would you debug which chain matches?

Enable narrow Spring Security debug/TRACE diagnostics and inspect FilterChainProxy's selected SecurityFilterChain; test route matching.

INTERVIEW QUESTION 008

What is a SecurityFilterChain bean?

EXPLAIN IN ONE BREATH

It pairs request-matching behavior with an ordered set of security filters for matching traffic.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@Bean
SecurityFilterChain web(HttpSecurity http) throws Exception {
http.authorizeHttpRequests(a -> a.anyRequest().authenticated())
.formLogin(Customizer.withDefaults());
return http.build();
}
```

Why it works: This browser-oriented sketch retains default CSRF behavior and assumes a configured user source. The DSL assembles filters; it is not a list of controller interceptors. Define the credential mechanism and authorization together.

⚠ **Interview trap:** Configuration snippets from different app types cannot be merged blindly.

FOLLOW-UP QUESTION — ANSWERED

What changes for a bearer-only API?

Use a dedicated matching API chain with oauth2ResourceServer JWT support, stateless context policy, narrow permissions, and reviewed CSRF/credential transport.

INTERVIEW QUESTION 009

How are multiple security chains selected?

EXPLAIN IN ONE BREATH

Only the first matching chain is selected, using chain order and its securityMatcher; a fallback prevents uncovered requests.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@Order(1) API chain: securityMatcher("/api/**")
@Order(2) Web chain: any remaining request
// Request /api/orders -> API chain only, not both chains.
```

Why it works: Rules in a later chain do not repair a permissive earlier match. Keep narrow chains first and a deliberate catch-all last. Test endpoints outside the expected prefixes.

⚠ **Interview trap:** A request matching no chain can bypass security-chain protections.

FOLLOW-UP QUESTION — ANSWERED

How would you detect a missing fallback?

Send a request outside every named chain and assert it is denied by a final catch-all chain; include unknown paths and verbs.

INTERVIEW QUESTION 010

How do securityMatcher and requestMatchers differ?

EXPLAIN IN ONE BREATH

securityMatcher chooses whether a chain applies; authorization requestMatchers choose rules inside the selected chain.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
http.securityMatcher("/api/**")
.authorizeHttpRequests(a -> a
.requestMatchers("/api/public/**").permitAll()
.anyRequest().authenticated());
```

Why it works: /api/public belongs to this chain but receives a public authorization rule. /web/home does not belong to this chain at all and needs another matching chain. The two matcher levels have different scope.

⚠ **Interview trap:** anyRequest inside one chain does not protect unmatched chains globally.

FOLLOW-UP QUESTION — ANSWERED

Where should a catch-all deny rule live?

Put a last, lowest-priority catch-all chain with anyRequest().denyAll() so unmatched routes are not exposed.

INTERVIEW QUESTION 011

Why does security filter order matter?

EXPLAIN IN ONE BREATH

A filter can act only on state prepared upstream; credential validation normally precedes authorization using that identity.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Load context -> exploit protection -> authentication
-> exception translation around downstream authorization
-> authorization -> application
```

Why it works: This is a conceptual sequence, not an exhaustive fixed filter list. Actual filters vary by enabled capabilities and version. Place custom filters relative to a known relevant framework filter.

⚠ **Interview trap:** Hard-coded numeric positions and copied full lists are brittle.

FOLLOW-UP QUESTION — ANSWERED

What information must a custom authorization filter already have?

It must receive a validated Authentication (identity and authorities) and trusted resource context before deciding permission.

INTERVIEW QUESTION 012

What does ExceptionTranslationFilter do?

EXPLAIN IN ONE BREATH

It translates relevant downstream authentication and access-denied failures into the configured security response path.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authentication required -> AuthenticationEntryPoint
Authenticated access denied -> AccessDeniedHandler
// Authentication filters also have their own failure handling.
```

Why it works: It is a bridge from exceptions to HTTP security behavior, not the component that validates a password. A failure raised before it in the chain is not automatically caught by it.

⚠ **Interview trap:** @ControllerAdvice does not automatically handle servlet security failures.

FOLLOW-UP QUESTION — ANSWERED

How should a custom authentication filter report failure?

Handle AuthenticationException through the filter's configured failure handler or AuthenticationEntryPoint and stop further processing.

03

Authentication model and context

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Login token

Provider decision

Context lifecycle

013 · What does Authentication represent?

014 · How do AuthenticationManager, ProviderManager and AuthenticationProvider differ?

015 · What is SecurityContextHolder?

016 · What does SecurityContextRepository do?

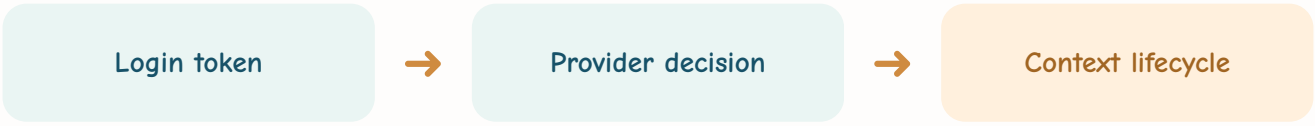
INTERVIEW QUESTION 013

What does Authentication represent?

EXPLAIN IN ONE BREATH

It can represent an authentication request or an authenticated result, carrying principal, credentials, authorities and status.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authentication request = UsernamePasswordAuthenticationToken
.unauthenticated("sai", "secret");
Authentication result = authenticationManager.authenticate(request);
```

Why it works: The manager delegates validation and returns the approved identity if successful. User-controlled data must not be marked trusted simply to bypass validation. A result's isAuthenticated state alone is not an object-level permission policy.

⚠ **Interview trap:** Application code should not fabricate trusted authorities from input.

FOLLOW-UP QUESTION — ANSWERED

What happens on an unsupported token type?

If no provider supports the token, ProviderManager cannot authenticate it and throws an appropriate authentication failure.

INTERVIEW QUESTION 014

How do AuthenticationManager, ProviderManager and AuthenticationProvider differ?

EXPLAIN IN ONE BREATH

AuthenticationManager is the entry contract; ProviderManager delegates to suitable providers; each provider validates supported authentication types.

QUICK VISUAL — WHAT HAPPENS?

Login token → Provider decision → Context lifecycle

ORIGINAL EXAMPLE / CODE

```
Authentication request -> ProviderManager
-> provider.supports(request class)
-> provider.authenticate(request)
-> authenticated result or failure
```

Why it works: A provider can return null when it does not handle that request. Several providers can coexist. A custom provider should validate its mechanism and return authorities from a trusted source.

⚠ **Interview trap:** supports is not the same as successful credential verification.

FOLLOW-UP QUESTION — ANSWERED

How would you combine password and another authentication mechanism?

Register supported providers for each mechanism; each authenticates only tokens it understands and returns a trusted result.

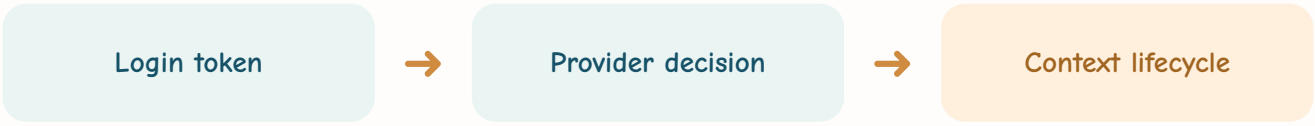
INTERVIEW QUESTION 015

What is SecurityContextHolder?

EXPLAIN IN ONE BREATH

It exposes the current SecurityContext using a configured strategy; the servlet default associates it with the executing thread.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authentication current = SecurityContextHolder.getContext().getAuthentication();
// Framework processing owns loading and clearing during requests.
```

Why it works: The context contains the current authentication. Manually setting it changes the current execution's identity but does not automatically persist a login for later requests. Keep lifecycle and trusted construction explicit.

⚠ **Interview trap:** ThreadLocal context does not automatically follow executor tasks.

FOLLOW-UP QUESTION — ANSWERED

Which repository persists servlet context?

A session-oriented implementation may use HttpSessionSecurityContextRepository; choose the repository deliberately.

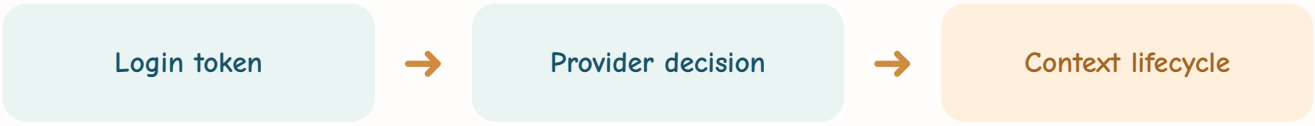
INTERVIEW QUESTION 016

What does SecurityContextRepository do?

EXPLAIN IN ONE BREATH

It loads and saves security context according to the application's persistence policy, such as session or request-only storage.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Context repository -> load existing context
Authentication succeeds -> save if persistent login is intended
Request completes -> thread-associated state cleared
```

Why it works: In modern explicit-save configurations, custom login code must save the context deliberately. Built-in mechanisms coordinate their supported persistence behavior. Do not rely on an old automatic-save assumption across versions.

⚠ **Interview trap:** Setting the holder is not sufficient proof that session login survives.

FOLLOW-UP QUESTION — ANSWERED

How does a stateless repository choice change the design?

A request-only or null repository does not preserve a login between requests; authenticate every request or supply a different persistence policy.

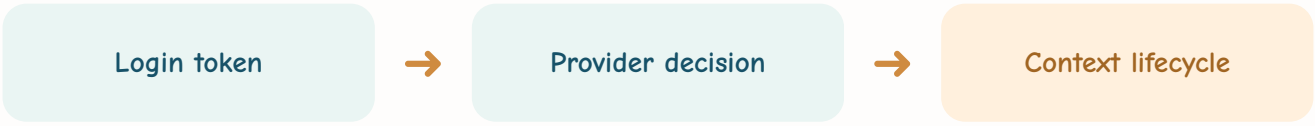
INTERVIEW QUESTION 017

Why does anonymous authentication not mean a logged-in user?

EXPLAIN IN ONE BREATH

It supplies a placeholder identity for anonymous processing, not verified end-user credentials.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authentication a = context.getAuthentication();
// An AnonymousAuthenticationToken can exist when no login occurred.
// Use the appropriate trust resolver or authorization API.
```

Why it works: Checking only a non-null Authentication or its raw authenticated flag can misclassify anonymous requests. Spring authorization helpers understand the relevant trust distinction. Use the configured access policy instead of ad hoc identity heuristics.

⚠ **Interview trap:** 'Authentication object exists' is not a login guarantee.

FOLLOW-UP QUESTION — ANSWERED

How do authenticated and fullyAuthenticated differ with remember-me?

authenticated() normally includes remember-me authentication, while fullyAuthenticated() requires stronger, non-remember-me authentication.

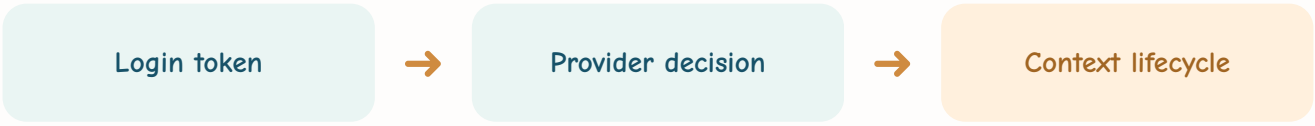
INTERVIEW QUESTION 018

How should security context propagate to async work?

EXPLAIN IN ONE BREATH

Use supported propagation wrappers or explicit trusted context with bounded ownership, then clean up worker state.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Executor delegated = new DelegatingSecurityContextExecutor(executor);
delegated.execute(() -> protectedService.run());
// Choose captured identity and task lifetime deliberately.
```

Why it works: The wrapper manages context around a task; it is not a durable authorization grant forever. For background work, recording an approved business command can be safer than retaining a user's live credentials.

⚠ **Interview trap:** InheritableThreadLocal is not a general solution for reused pools.

FOLLOW-UP QUESTION — ANSWERED

How would you prevent stale context between tasks?

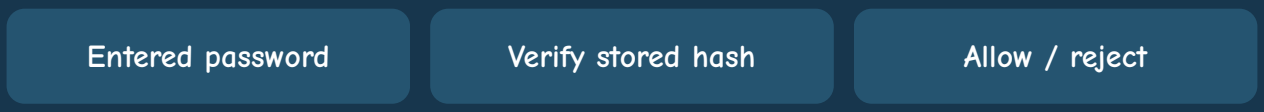
Use DelegatingSecurityContextExecutor wrappers or scoped setup; always restore/clear context in finally and test executor reuse.

04

Password authentication and user storage

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL



019 · What is UserDetailsService responsible for?

020 · How does DaoAuthenticationProvider use PasswordEncoder?

021 · What does DelegatingPasswordEncoder add?

022 · Why is plain hashing with SHA-256 insufficient for passwords?

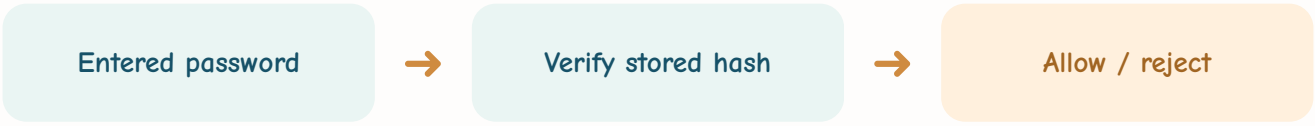
INTERVIEW QUESTION 019

What is UserDetailsService responsible for?

EXPLAIN IN ONE BREATH

It loads user identity data, password representation, authorities and account flags; it does not itself implement the entire password-authentication flow.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
UserDetails loadUserByUsername(String username) {
Account a = store.findNormalizedUsername(username).orElseThrow();
return User.withUsername(a.username()).password(a.passwordHash())
.authorities(a.authorities()).build();
}
```

Why it works: The sketch must translate absence into the supported UsernameNotFoundException and preserve account-status rules. A password provider uses the loaded data with its encoder. Do not expose the stored hash in an API response.

⚠ **Interview trap:** A database entity is not automatically the correct public principal.

FOLLOW-UP QUESTION — ANSWERED

How would disabled accounts be represented?

Expose enabled=false in UserDetails or equivalent account policy; authentication must reject disabled accounts.

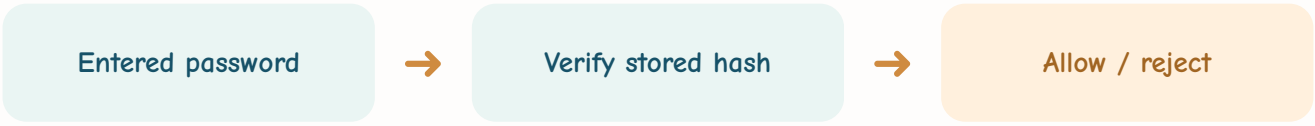
INTERVIEW QUESTION 020

How does DaoAuthenticationProvider use PasswordEncoder?

EXPLAIN IN ONE BREATH

It obtains user details and verifies supplied credentials against the stored adaptive-hash representation through the configured encoder.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
PasswordEncoder encoder = PasswordEncoderFactories.createDelegatingPasswordEncoder();
String stored = encoder.encode("example");
System.out.println(encoder.matches("example", stored)); // true
```

Why it works: Salts mean two encode calls can produce different stored strings for the same password. Verify with matches, not string equality after re-encoding. Use the supported provider configuration for the exact baseline.

⚠ **Interview trap:** Do not log the supplied password during authentication.

FOLLOW-UP QUESTION — ANSWERED

Why should cost be measured on production-class hardware?

A hashing cost that is safe on a laptop may overload production CPU; benchmark login latency and throughput on representative hardware.

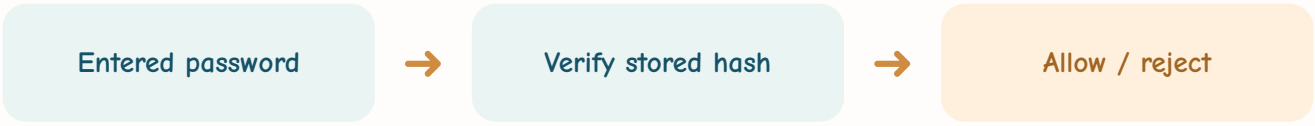
INTERVIEW QUESTION 021

What does DelegatingPasswordEncoder add?

EXPLAIN IN ONE BREATH

It selects a password-hash implementation from an identifier prefix, supporting verification and algorithm evolution.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
{bcrypt}$2a$... -> bcrypt verification
{argon2}...     -> configured Argon2 verification
// Exact encoder ids and implementation availability depend on setup.
```

Why it works: The prefix describes the stored format rather than encrypting the password. Supported upgrade detection can trigger rehashing after successful authentication using a user-details password update service.

⚠ **Interview trap:** Missing or unknown ids need a deliberate migration policy.

FOLLOW-UP QUESTION — ANSWERED

How would you upgrade an older hash safely?

On successful login, verify the old hash, check upgradeEncoding, and atomically store a new adaptive hash without ever storing plaintext.

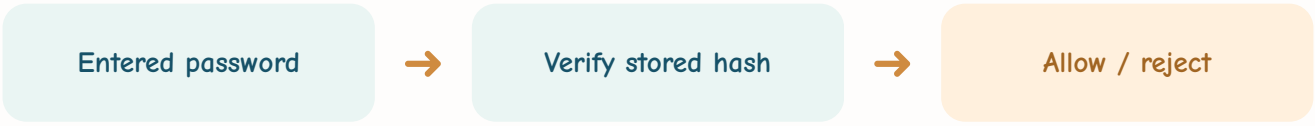
INTERVIEW QUESTION 022

Why is plain hashing with SHA-256 insufficient for passwords?

EXPLAIN IN ONE BREATH

Fast general-purpose hashes are cheap to guess at scale; use salted adaptive password hashing with an appropriate cost and policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Bad: SHA256(password) as a login database scheme
Better: supported adaptive PasswordEncoder + measured cost
Store hash, never reversible raw-password recovery data.
```

Why it works: Salt prevents identical-password equality from being visible through identical unsalted hashes. Cost slows offline guessing. Pick a reviewed algorithm and implementation; do not invent a custom cryptographic scheme.

⚠ **Interview trap:** Encoding, hashing and encryption solve different problems.

FOLLOW-UP QUESTION — ANSWERED

What additional control slows online guessing?

Apply per-account/IP/device rate limits, progressive delays, abuse detection and MFA as appropriate.

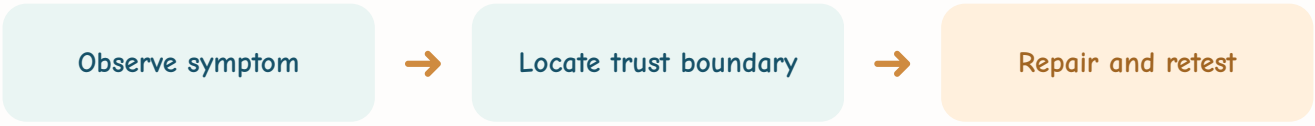
INTERVIEW QUESTION 023

How should authentication failures avoid user enumeration?

EXPLAIN IN ONE BREATH

Use appropriately uniform public responses and timing policy while retaining safe internal diagnostics and abuse monitoring.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Unknown username -> generic public login failure
Wrong password   -> same public failure contract
Internal event   -> safe category for monitoring, no password
```

Why it works: Different wording, status, recovery flows or timing can disclose account existence. Balance usable recovery with abuse resistance. A uniform message alone does not eliminate every side channel.

⚠ **Interview trap:** Logging the entire login request can expose credentials.

FOLLOW-UP QUESTION — ANSWERED

How would reset-password responses follow the same policy?

Return the same public acknowledgement whether the account exists, and send recovery instructions only through its verified channel.

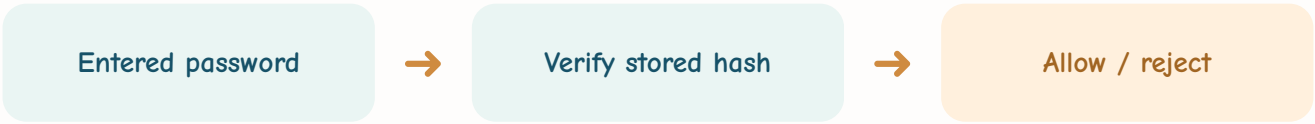
INTERVIEW QUESTION 024

Which account-state checks matter beyond password validity?

EXPLAIN IN ONE BREATH

Check enabled, locked, account expiry, credential expiry, and any application-specific authentication policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Valid password + disabled account -> refuse login
Valid password + required second factor -> incomplete assurance
Permission change -> update authority and session/token policy
```

Why it works: UserDetails flags cover common status conditions, while business assurance and factor policy may need additional design. Do not equate a correct password with unlimited access.

⚠ **Interview trap:** Account disablement may not immediately revoke an already-issued self-contained token.

FOLLOW-UP QUESTION — ANSWERED

Which component rechecks active sessions or tokens?

A session/token validation or central revocation/authorization layer must recheck status according to the required latency; issuing a token does not.

Q025–Q030

05

Sessions, login, logout and remember-me

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Browser credentials

Persistent session

Logout / expiry

025 · How does form login differ from HTTP Basic?

026 · What does SessionCreationPolicy.STATELESS guarantee?

027 · How does session fixation protection help?

028 · How should logout behave for a session application?

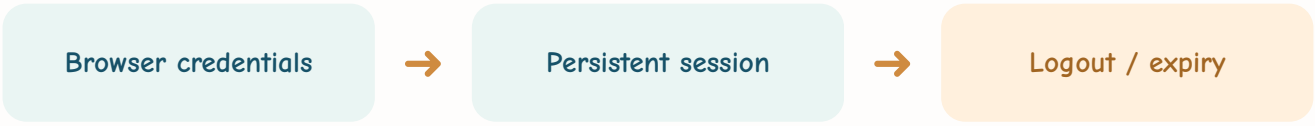
INTERVIEW QUESTION 025

How does form login differ from HTTP Basic?

EXPLAIN IN ONE BREATH

Form login commonly establishes a browser session; Basic sends username/password credentials encoded in each applicable request.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Form: login submission -> authenticated session cookie
Basic: Authorization: Basic <base64(username:password)>
// Base64 is readable encoding, not encryption.
```

Why it works: Require TLS for credential transport and choose client behavior deliberately. Browsers can automatically resend Basic credentials, which affects CSRF analysis. Neither mechanism is itself multi-factor authentication.

⚠ **Interview trap:** Do not call Basic secure merely because the value is encoded.

FOLLOW-UP QUESTION — ANSWERED

Why might an API use a bearer token instead?

Bearer tokens avoid resending the password to every API; validate issuer, signature, audience, expiry and scopes for each request.

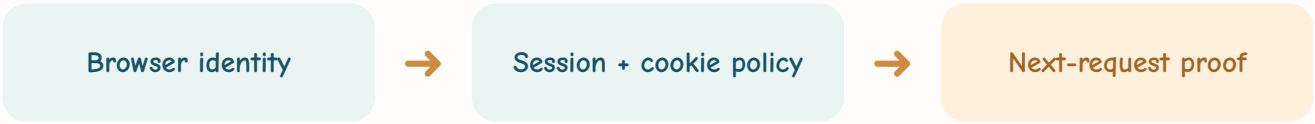
INTERVIEW QUESTION 026

What does SessionCreationPolicy.STATELESS guarantee?

EXPLAIN IN ONE BREATH

Spring Security does not create or use an HttpSession to obtain its SecurityContext under that policy; unrelated application code can still create a session.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
http.sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
// Intended context strategy: authenticate bearer requests separately.
```

Why it works: The setting controls security's session behavior, not a universal ban on every cookie or session in the application. Review request caching and other stateful mechanisms separately.

⚠ **Interview trap:** Stateless does not automatically mean CSRF-safe.

FOLLOW-UP QUESTION — ANSWERED

How would a servlet controller still create a session?

Code can still call request.getSession() or request.getSession(true); Security's stateless setting does not ban that servlet operation.

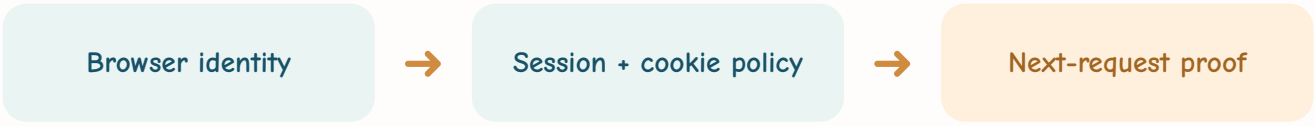
INTERVIEW QUESTION 027

How does session fixation protection help?

EXPLAIN IN ONE BREATH

It changes or replaces the session identifier/state at authentication according to supported policy, preventing reuse of a pre-login identifier.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Attacker knows pre-login session id
Victim authenticates -> session id changes
Attacker's old identifier should not grant the authenticated session.
```

Why it works: Modern servlet integration can use changeSessionId. Exact strategy and session-attribute behavior depend on configuration. Validate the actual login flow when implementing custom authentication.

⚠ **Interview trap:** Rotating the id does not prevent every stolen-session attack.

FOLLOW-UP QUESTION — ANSWERED

What happens to concurrent tabs with the same browser session?

Tabs sharing the browser's session cookie generally follow the rotated session identifier; verify custom clients and in-flight requests.

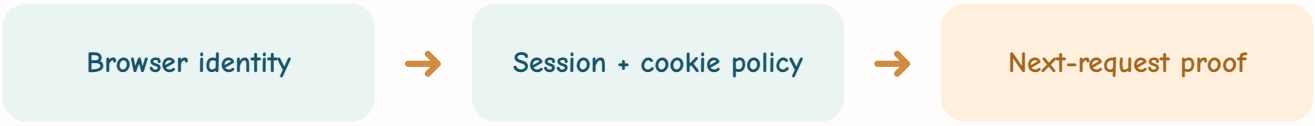
INTERVIEW QUESTION 028

How should logout behave for a session application?

EXPLAIN IN ONE BREATH

Use the framework logout flow to clear security state and invalidate the relevant session, with CSRF protection for state-changing browser behavior.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
<form method="post" action="/logout">
<!-- Include framework CSRF token field. -->
<button>Log out</button>
</form>
```

Why it works: The exact endpoint and success response can be configured. Cookie cleanup and remember-me cleanup should match your setup. A GET route that merely displays confirmation differs from the actual state-changing logout.

⚠ **Interview trap:** Local logout does not automatically log out an identity provider.

FOLLOW-UP QUESTION — ANSWERED

How does logout differ for self-contained bearer tokens?

Session logout invalidates the server session and clears authentication; JWT logout needs short lifetimes or a separate revocation mechanism.

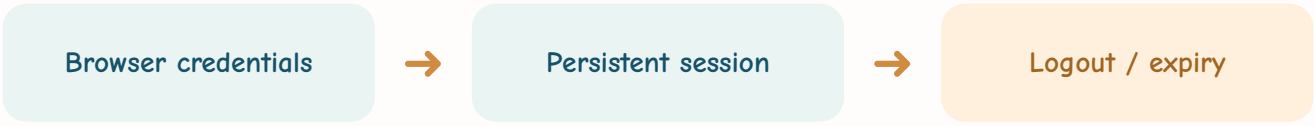
INTERVIEW QUESTION 029

What is remember-me authentication and what is its limitation?

EXPLAIN IN ONE BREATH

It provides longer-lived browser reauthentication through a cookie-based policy; it is weaker than fresh full authentication for sensitive actions.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Remember-me cookie -> restored identity
Sensitive password change -> require fresh authentication/MFA
// fullyAuthenticated can distinguish supported remember-me state.
```

Why it works: Token storage, expiry, theft detection and revocation differ by implementation. Do not confuse remember-me with an OAuth refresh token. Keep sensitive-operation assurance explicit.

⚠ **Interview trap:** A remembered identity is not proof the password was just supplied.

FOLLOW-UP QUESTION — ANSWERED

How would password changes affect remember-me tokens?

Invalidate or rotate persistent remember-me tokens when passwords change; server-side token records are easier to revoke.

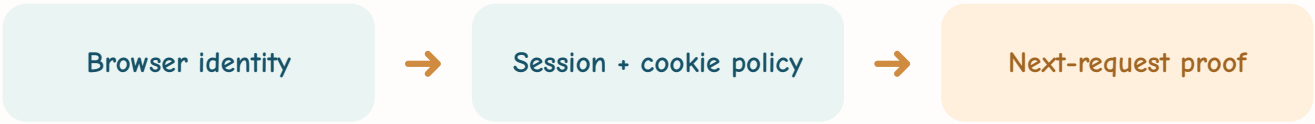
INTERVIEW QUESTION 030

How can concurrent sessions be limited?

EXPLAIN IN ONE BREATH

Configure the session-control mechanism and its registry for the selected login flow, then define whether new logins replace or reject old sessions.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
http.sessionManagement(s -> s.maximumSessions(1));
// Complete setup must match the application and session registry.
```

Why it works: A cluster needs a distributed-session/registry strategy where required. Custom authentication must apply relevant session strategies. Browser session limits do not universally restrict OAuth token issuance.

⚠ **Interview trap:** Process-local session tracking does not automatically cover all replicas.

FOLLOW-UP QUESTION — ANSWERED

Which session should remain after a second login?

Choose a policy (reject new, expire old, or notify), test it explicitly, and explain what happens to the former session.

06

HTTP authorization and chain boundaries

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Request + method

Matcher + authority

Allow / deny

031 · How are authorization request rules selected?

032 · How does permitAll differ from ignoring security processing?

033 · Why should authorization consider the HTTP method?

034 · What is deny-by-default and why use it?

INTERVIEW QUESTION 031

How are authorization request rules selected?

EXPLAIN IN ONE BREATH

The first matching authorization rule in the selected chain is applied; order specific rules before broad ones.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
http.authorizeHttpRequests(a -> a
.requestMatchers("/admin/**").hasRole("ADMIN")
.requestMatchers("/public/**").permitAll()
.anyRequest().authenticated());
```

Why it works: A broad permitAll placed first can make narrower restrictions unreachable. This rule ordering is separate from security-chain ordering. Inspect both levels when a protected route unexpectedly succeeds.

⚠ **Interview trap:** anyRequest is normally the terminal fallback rule.

FOLLOW-UP QUESTION — ANSWERED

How would you test an unexpected route?

Test a new unmatched URL and an unexpected HTTP method; require a specific allow rule or a deny-all fallback.

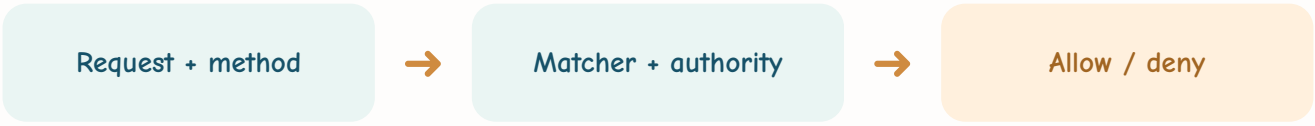
INTERVIEW QUESTION 032

How does permitAll differ from ignoring security processing?

EXPLAIN IN ONE BREATH

permitAll allows authorization within the chain; ignoring bypasses the selected security protections for that request.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
/public/** -> permitAll, security headers can still apply
Ignored path -> security chain bypassed for that path
```

Why it works: Prefer permitAll when public routes should retain exploit protection and headers. An ignored route cannot rely on that chain to populate or protect its context. Review static-resource policy deliberately.

⚠ **Interview trap:** 'Public' need not mean 'outside all security filters'.

FOLLOW-UP QUESTION — ANSWERED

Why can a permitted POST still fail CSRF?

permitAll skips authorization denial, not CSRF validation for unsafe browser requests; a POST can still be rejected.

INTERVIEW QUESTION 033

Why should authorization consider the HTTP method?

EXPLAIN IN ONE BREATH

Read and write operations on the same path can require different permissions.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
.requestMatchers(HttpMethod.GET, "/orders/**").hasAuthority("orders:read")
.requestMatchers(HttpMethod.POST, "/orders/**").hasAuthority("orders:create")
.anyRequest().denyAll()
```

Why it works: Route shape alone does not distinguish a safe read from a modifying operation. Handle preflight and other dispatches in their correct mechanism. Backend ownership checks still apply.

⚠ **Interview trap:** Method-specific rules do not prevent unsafe business semantics inside GET.

FOLLOW-UP QUESTION — ANSWERED

What permission should a bulk export require?

Bulk export should require a dedicated, least-privilege permission plus row/tenant filtering and audit logging.

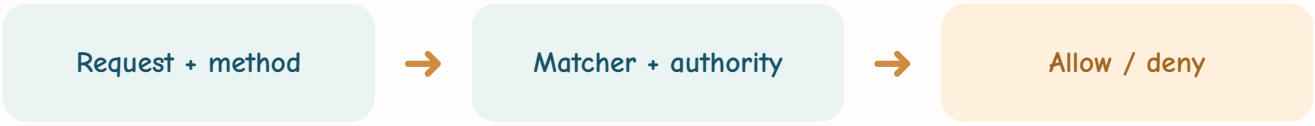
INTERVIEW QUESTION 034

What is deny-by-default and why use it?

EXPLAIN IN ONE BREATH

Unspecified operations are refused rather than accidentally opened as the application grows.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
http.authorizeHttpRequests(a -> a
.requestMatchers("/health/public").permitAll()
.requestMatchers("/api/orders/**").hasAuthority("orders:read")
.anyRequest().denyAll());
```

Why it works: New endpoints need an explicit policy instead of inheriting unrestricted access. Test framework endpoints, error dispatches and static paths that should work intentionally. Deny-by-default is useful only if rules express the real operation semantics.

⚠ **Interview trap:** An overly broad allowed prefix can still expose new sensitive routes.

FOLLOW-UP QUESTION — ANSWERED

How would a new endpoint fail a security regression test?

A new endpoint should fail closed under anyRequest().denyAll(), and a regression test should assert anonymous and underprivileged denial.

INTERVIEW QUESTION 035

How do authenticated and fullyAuthenticated differ?

EXPLAIN IN ONE BREATH

authenticated can include supported remembered authentication; fullyAuthenticated excludes anonymous and remember-me authentication states.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Public page -> permitAll
Account overview -> authenticated
Sensitive credential change -> fresh assurance policy
```

Why it works: A helper expresses one trust distinction, not a complete MFA or recent-login policy. If recent authentication is required, check the actual assurance and time evidence from a trusted mechanism.

⚠ **Interview trap:** fullyAuthenticated is not universally equivalent to MFA.

FOLLOW-UP QUESTION — ANSWERED

Where would step-up authentication be recorded?

Record assurance level and authentication time in trusted session/token context, then enforce step-up on sensitive actions.

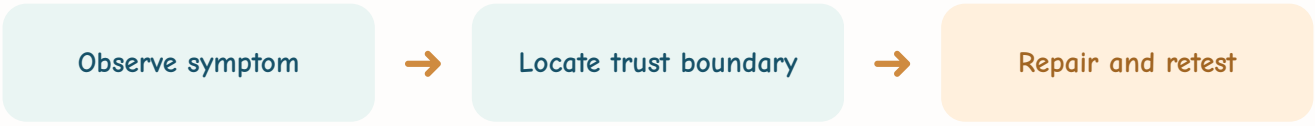
INTERVIEW QUESTION 036

How should ERROR and ASYNC dispatches be considered?

EXPLAIN IN ONE BREATH

Security matching can include dispatcher types, so test secondary dispatches and intentionally choose their access policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
.requestMatchers(DispatcherType.ERROR, DispatcherType.FORWARD).permitAll()
// Illustrative choice: review error/forward behavior before adopting.
```

Why it works: An original authorized request can encounter another check during rendering or error handling. Avoid masking the original error with an unexpected denial, while ensuring forwards cannot reach sensitive operations unchecked.

⚠ **Interview trap:** Blanket dispatch exemptions require an actual boundary analysis.

FOLLOW-UP QUESTION — ANSWERED

How would a protected forwarded handler be tested?

Test FORWARD/ERROR/ASYNC dispatches to protected handlers, checking that authorization still prevents forbidden effects.

07

Method and object-level security

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Actor + object

Ownership check

Guard business effect

037 · How is method security enabled?

038 · How do @PreAuthorize and @PostAuthorize differ?

039 · What do @PreFilter and @PostFilter do?

040 · How can custom permission logic be used in expressions?

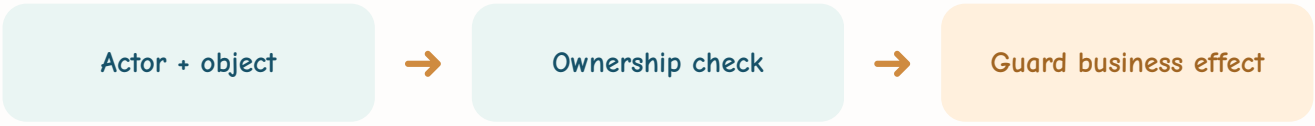
INTERVIEW QUESTION 037

How is method security enabled?

EXPLAIN IN ONE BREATH

Use @EnableMethodSecurity and supported annotations on eligible Spring-managed method boundaries.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@Configuration
@EnableMethodSecurity
class MethodSecurityConfig {}
@PreAuthorize("hasAuthority('orders:approve')")
public void approve(long id) { /* domain work */ }
```

Why it works: The call must cross the advice boundary. Enabling servlet request security does not automatically enable every method-security annotation. Check proxy eligibility and invocation paths.

⚠ **Interview trap:** Self-invocation can bypass ordinary proxy advice.

FOLLOW-UP QUESTION — ANSWERED

Which callers might bypass a controller's URL rule?

Service methods invoked by jobs, messages, or other controllers can bypass a particular controller URL rule; enforce reusable business authorization.

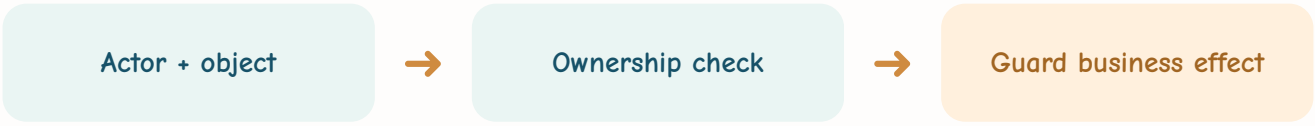
INTERVIEW QUESTION 038

How do @PreAuthorize and @PostAuthorize differ?

EXPLAIN IN ONE BREATH

PreAuthorize checks before invocation; PostAuthorize checks afterward and can inspect the returned object.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@PostAuthorize("returnObject.owner == authentication.name")
public OrderView load(long id) { return store.loadView(id); }
```

Why it works: Post-authorization can prevent returning a result but cannot undo arbitrary side effects already performed. Use prechecks or authoritative tenant-scoped operations for modifying methods. Query-level ownership can also avoid loading unrelated data.

⚠ **Interview trap:** PostAuthorize is unsuitable as the only guard on an irreversible write.

FOLLOW-UP QUESTION — ANSWERED

What happens to a null or Optional return shape?

A null/empty Optional result may affect the expression; define and test the actual return shape, especially for @PostAuthorize.

INTERVIEW QUESTION 039

What do @PreFilter and @PostFilter do?

EXPLAIN IN ONE BREATH

They filter supported input or output collections according to per-element expressions, rather than making every element safe automatically.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@PostFilter("filterObject.owner == authentication.name")
public List<OrderView> list() { return store.findCandidateViews(); }
```

Why it works: Filtering can remove unauthorized results, but loading all candidates may be expensive and counts/pagination can leak or mislead. Prefer appropriately scoped queries when possible. The returned type must be supported for the selected annotation.

⚠ **Interview trap:** Filtering an in-memory page can produce inconsistent totals.

FOLLOW-UP QUESTION — ANSWERED

How would row ownership be enforced in the query?

Apply tenant and ownership constraints in the database query itself, not only by filtering a large returned collection.

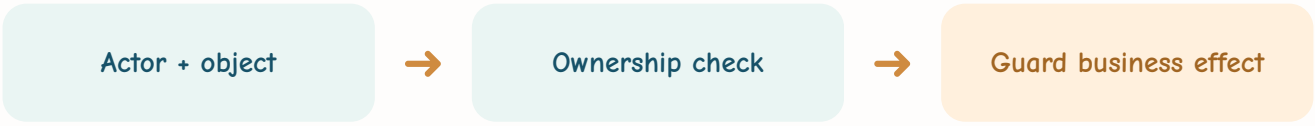
INTERVIEW QUESTION 040

How can custom permission logic be used in expressions?

EXPLAIN IN ONE BREATH

Delegate complex decisions to a trusted policy bean or supported permission evaluator rather than packing them into fragile strings.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@PreAuthorize("@orderPolicy.canRead(authentication, #id)")
public OrderView read(long id) { return store.read(id); }
```

Why it works: The policy checks the trusted identity and authoritative ownership. Preserve method parameter names or use a supported explicit naming approach. Keep policy decisions testable without granting privileges from request input.

⚠ **Interview trap:** Repeating a remote permission lookup can create latency and availability coupling.

FOLLOW-UP QUESTION — ANSWERED

What data should a policy cache include?

Include principal, tenant, effective permissions, resource/version, and policy-version/expiry in the cache key or invalidate on change.

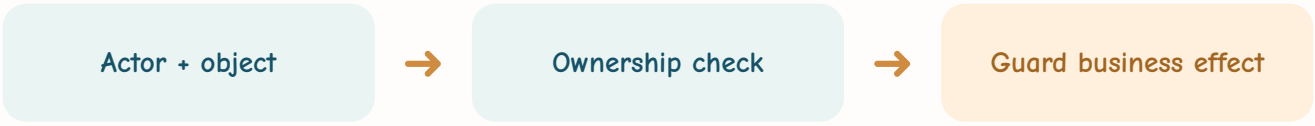
INTERVIEW QUESTION 041

How do @Secured and JSR-250 annotations compare with expressions?

EXPLAIN IN ONE BREATH

They provide simpler role-oriented declarations; expression-based annotations support richer predicates and bean calls.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@EnableMethodSecurity(securedEnabled=true, jsr250Enabled=true)
@Secured("ROLE_ADMIN")
public void archive() { /* protected work */ }
```

Why it works: The simpler annotations need their corresponding enablement. Boot 3's JSR-250 namespace is Jakarta where applicable. Choose one clear policy style rather than overlapping contradictory checks casually.

⚠ **Interview trap:** Enabling method security does not necessarily enable every optional annotation family.

FOLLOW-UP QUESTION — ANSWERED

When would a custom composed annotation help?

A composed annotation helps when many endpoints need the same stable permission policy; verify its meta-annotation behavior.

INTERVIEW QUESTION 042

What is IDOR or broken object-level authorization?

EXPLAIN IN ONE BREATH

It occurs when changing an object identifier lets a caller access another user's or tenant's resource without the required permission check.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
GET /invoices/7 -> caller-owned invoice
GET /invoices/8 -> another tenant's invoice
Query: WHERE id=:id AND tenant_id=:trustedTenant
```

Why it works: A valid role is not sufficient if ownership is part of the contract. Derive tenant identity from trusted authentication, then enforce it in lookup and modification. Avoid distinguishing forbidden existence where policy forbids disclosure.

⚠ **Interview trap:** Random UUIDs are not an authorization policy.

FOLLOW-UP QUESTION — ANSWERED

How would a message consumer enforce the same rule?

A consumer must validate the trusted initiator/tenant and resource ownership before executing a message's business effect.

08

CSRF and browser credential behavior

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Browser sends cookie

CSRF token check

Unsafe action protected

043 · What is CSRF?

044 · How does a synchronizer CSRF token help?

045 · When can a bearer-only API disable CSRF deliberately?

046 · Does SameSite eliminate the need for CSRF protection?

INTERVIEW QUESTION 043

What is CSRF?

EXPLAIN IN ONE BREATH

It is unwanted state-changing action induced through a victim's browser using credentials the browser supplies automatically.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Victim logged in with session cookie
Malicious site triggers a modifying request
Browser may attach victim's credential -> server needs CSRF defense
```

Why it works: The attacker need not read the response to cause the action. CSRF tokens establish a request-bound proof unavailable to an unrelated origin under the expected browser model. XSS can undermine that model.

⚠ **Interview trap:** CORS restrictions alone do not prevent every cross-site submission.

FOLLOW-UP QUESTION — ANSWERED

Why must GET avoid state-changing semantics?

GET must remain safe: browsers and intermediaries can prefetch or repeat it, so do not charge, delete or modify on GET.

INTERVIEW QUESTION 044

How does a synchronizer CSRF token help?

EXPLAIN IN ONE BREATH

The server checks a token associated with the legitimate session/request flow for unsafe methods, in addition to the automatically sent credential.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
<form method="post" action="/orders">
<input type="hidden" name="_csrf" value="SERVER_TOKEN">
</form>
```

Why it works: Use the actual framework-provided parameter/header name and token, not the literal placeholder. Keep tokens out of logs and URLs. Missing or invalid tokens follow the CSRF failure path.

⚠ **Interview trap:** A static shared token provides no meaningful per-client binding.

FOLLOW-UP QUESTION — ANSWERED

How does a JavaScript client obtain the token safely?

Expose a token safely via approved token endpoint or cookie/header mechanism, then send it in a request header; guard against XSS.

INTERVIEW QUESTION 045

When can a bearer-only API disable CSRF deliberately?

EXPLAIN IN ONE BREATH

When its credential model does not rely on browser-automatically attached credentials, a dedicated bearer-header-only chain can use that policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
API accepts only Authorization: Bearer <access-token>
No session-cookie, Basic, or token-cookie authentication in this chain
Then assess .csrf(csrf -> csrf.disable()) for that boundary.
```

Why it works: The decision follows credential transport, not the word REST or stateless. Reassess if browser cookies or additional authentication mechanisms are introduced. Keep another browser chain's CSRF protections intact.

⚠ **Interview trap:** JWT in an auto-sent cookie remains a different CSRF case.

FOLLOW-UP QUESTION — ANSWERED

What would a regression test assert after adding cookie login?

After adding cookie login, a regression test must send a modifying request without a CSRF token and assert denial/no business effect.

INTERVIEW QUESTION 046

Does SameSite eliminate the need for CSRF protection?

EXPLAIN IN ONE BREATH

It is useful defense in depth, but its mode, browser behavior and application flows must be considered; do not treat it as a universal substitute.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Session cookie: Secure; HttpOnly; SameSite=Lax
Cross-site workflow -> review redirects, form submissions and site boundaries
Unsafe action -> framework CSRF policy remains deliberate
```

Why it works: Same-site and same-origin are different concepts, and sibling subdomains can matter. Some applications need cross-site cookie behavior. Validate the entire flow rather than copying flags without testing.

⚠ **Interview trap:** SameSite=None requires a corresponding secure-cookie policy in modern browsers.

FOLLOW-UP QUESTION — ANSWERED

How would an embedded application affect cookie choice?

Embedded/third-party contexts may require SameSite=None; Secure and a stricter CSRF policy, with browser tests.

INTERVIEW QUESTION 047

How does XSS differ from CSRF?

EXPLAIN IN ONE BREATH

XSS executes attacker-controlled script in a trusted page context; CSRF abuses automatically supplied credentials across sites.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
XSS -> attacker script can act as the trusted origin
CSRF -> unrelated origin induces a credentialed request
```

Why it works: Output encoding, safe templating, sanitization and CSP help address XSS, while CSRF tokens address a different request-authenticity boundary. HttpOnly protects cookie reading but not every action malicious script can perform.

⚠ **Interview trap:** A stolen-token prevention flag is not full XSS prevention.

FOLLOW-UP QUESTION — ANSWERED

What can XSS do even with HttpOnly cookies?

XSS can issue requests as the user, inspect page contents, or steal non-HttpOnly tokens even if it cannot read HttpOnly cookies.

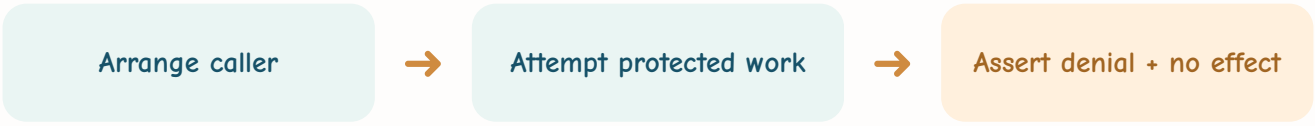
INTERVIEW QUESTION 048

What are important CSRF lifecycle cases to test?

EXPLAIN IN ONE BREATH

Test first load, unsafe requests, invalid token, authentication transition, logout, and renewed token state for the selected client integration.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Login/logout -> token state may be cleared or renewed
SPA -> obtain and send the correct current token
Missing/invalid token -> denied unsafe request
```

Why it works: Modern deferred-token and masking behavior affects how a client reads and sends tokens. Use the supported SPA configuration for the baseline instead of old cookie snippets. Test behavior, not merely cookie presence.

⚠ **Interview trap:** A stale token after login can cause persistent 403 responses.

FOLLOW-UP QUESTION — ANSWERED

Which endpoint safely refreshes the client token?

Use an authenticated safe endpoint with intended CSRF token repository/refresh behavior; check first load, login, logout and expiry.

09

CORS, headers and transport protection

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Origin + request

CORS + headers

Browser policy

049 · What does CORS control?

050 · Why should CORS processing precede security authentication for preflight?

051 · How should allowed origins and credentials be configured?

052 · What do common Spring Security response headers protect?

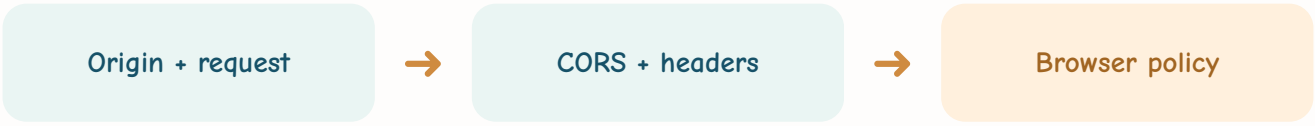
INTERVIEW QUESTION 049

What does CORS control?

EXPLAIN IN ONE BREATH

It governs browser cross-origin access according to server response policy; it is not caller authentication.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Origin: https://app.example
Allowed origin -> browser may accept permitted response access
curl or server-to-server client -> does not enforce browser CORS
```

Why it works: Authenticate and authorize the actual request independently. A denied CORS response does not prove the server never executed a simple request. Distinguish preflight prevention from response-read restrictions.

⚠ **Interview trap:** An Origin header is not trusted account identity.

FOLLOW-UP QUESTION — ANSWERED

Why can Postman succeed when browser JavaScript fails?

Postman does not enforce browser cross-origin policy; browser JavaScript enforces CORS when reading cross-origin responses.

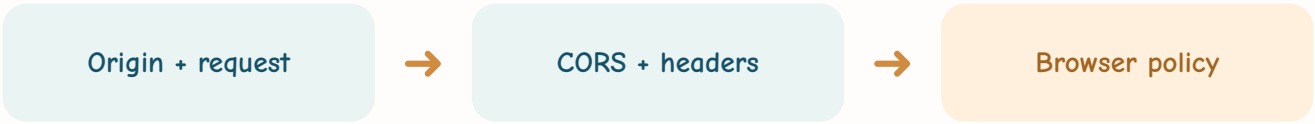
INTERVIEW QUESTION 050

Why should CORS processing precede security authentication for preflight?

EXPLAIN IN ONE BREATH

A preflight usually lacks the actual authentication credential, so it must receive the configured CORS decision before ordinary login requirements reject it.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
OPTIONS /api/orders
Access-Control-Request-Method: GET
Access-Control-Request-Headers: authorization
```

Why it works: Configure a CorsConfigurationSource or matching supported integration and enable CORS in the relevant chain. The following actual request still needs authentication and authorization.

⚠ **Interview trap:** Permitting preflight does not permit all business requests.

FOLLOW-UP QUESTION — ANSWERED

What headers must be returned for the allowed request?

The preflight response needs an allowed Origin, method, requested headers, and credentials permission only when configured.

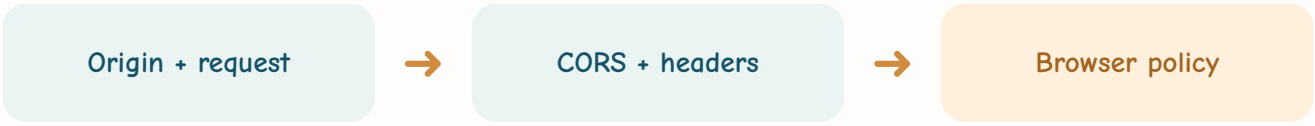
INTERVIEW QUESTION 051

How should allowed origins and credentials be configured?

EXPLAIN IN ONE BREATH

Use a precise trusted-origin policy and deliberately enable credentials only when required by the browser architecture.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Allowed origin: https://app.example
Allowed methods: GET, POST
Allowed headers: Authorization, Content-Type, X-XSRF-TOKEN
Allow credentials: choose from actual cookie/credential requirements
```

Why it works: Wildcard origins do not combine with credentialed policy in the same way as explicit origins. Avoid broadly reflecting arbitrary origins. Keep development localhost settings separate from production trust.

⚠ **Interview trap:** A wildcard domain pattern can admit unexpected tenants or hosts.

FOLLOW-UP QUESTION — ANSWERED

How would you test a denied Origin?

Assert a disallowed Origin never receives permissive Access-Control-Allow-Origin/ credentials headers, even on OPTIONS.

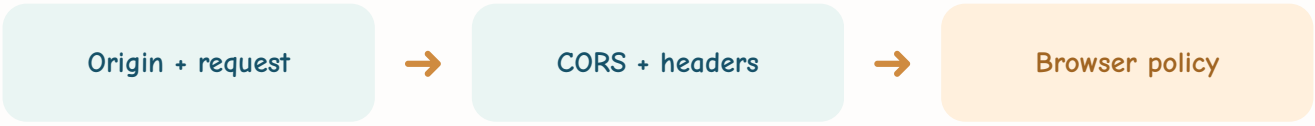
INTERVIEW QUESTION 052

What do common Spring Security response headers protect?

EXPLAIN IN ONE BREATH

They provide selected browser protections, including content-type sniffing, framing and transport behavior according to configuration.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY (where configured)
Strict-Transport-Security: HTTPS response policy
```

Why it works: Header defaults vary by feature and release. HSTS is normally emitted for secure requests, making reverse-proxy scheme handling important. A header only works where the browser applies its semantics.

⚠ **Interview trap:** Security headers do not authenticate an API caller.

FOLLOW-UP QUESTION — ANSWERED

How would a TLS-terminating proxy affect HSTS?

Terminate HTTPS at a trusted proxy, correctly signal secure request scheme, and set HSTS only for reviewed HTTPS hosts.

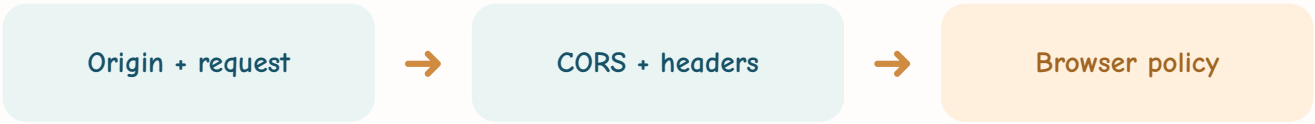
INTERVIEW QUESTION 053

What does Content Security Policy add?

EXPLAIN IN ONE BREATH

CSP restricts permitted resource/script behavior according to a deliberately configured browser policy, helping reduce selected injection impact.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'
// Tailor scripts, styles, nonce/hash rules and integrations.
```

Why it works: Spring Security does not know the application's asset policy automatically. Test in report-only mode where useful, then enforce reviewed rules. Avoid unsafe directives that undermine the intended protection.

⚠ **Interview trap:** CSP is not a replacement for contextual output encoding.

FOLLOW-UP QUESTION — ANSWERED

How would third-party scripts change the policy?

Limit allowed script sources and nonces/hashes; every third-party script expands the trusted code surface.

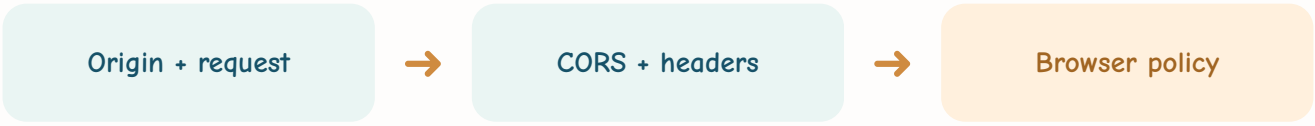
INTERVIEW QUESTION 054

How should trusted proxy headers be handled?

EXPLAIN IN ONE BREATH

Accept forwarded scheme/host/client information only through a trusted proxy boundary with sanitization and correct application configuration.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Public HTTPS -> trusted ingress -> application HTTP
Ingress replaces untrusted forwarded headers
Application resolves secure scheme from trusted metadata
```

Why it works: Forged forwarded host or scheme data can affect redirects, cookies, links and transport checks. Configure the selected Boot/server handling in conjunction with the network topology. Do not trust arbitrary internet-supplied forwarding headers.

⚠ **Interview trap:** Merely adding ForwardedHeaderFilter does not establish proxy trust.

FOLLOW-UP QUESTION — ANSWERED

How would you test an injected external redirect host?

Sanitize/validate trusted proxy headers; test Host/X-Forwarded-Host injection against absolute redirects and generated links.

10

JWT structure and cryptographic trust

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

JWT header + claims

Signature verification

Trust decision

055 · What is a JWT and what are its parts?

056 · How do JWS and JWE differ?

057 · How do symmetric and asymmetric JWT signatures differ?

058 · What do iss, sub, aud, exp and nbf mean?

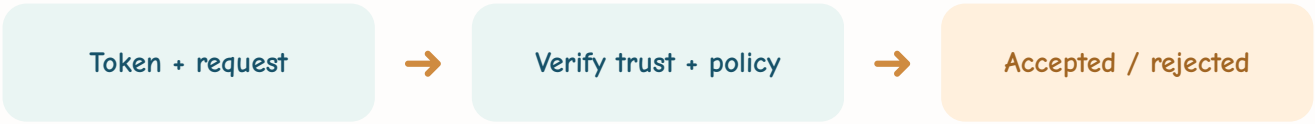
INTERVIEW QUESTION 055

What is a JWT and what are its parts?

EXPLAIN IN ONE BREATH

A common signed JWT uses encoded header, payload and signature segments; the claims are readable unless a separate encryption format is used.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
base64url(header).base64url(payload).signature
Header: algorithm and possible key id
Payload: claims such as iss, sub, aud, exp
```

Why it works: A signature verifies integrity and signer trust under the selected validation policy. Encoding does not hide claims. Not every JWT is signed in this exact compact form, so name the signed-token case.

⚠ **Interview trap:** Decoding a payload does not authenticate its contents.

FOLLOW-UP QUESTION — ANSWERED

Which claims should never contain secrets?

Avoid passwords, API keys, session IDs, personal information, or anything that must remain secret in signed-only token claims.

INTERVIEW QUESTION 056

How do JWS and JWE differ?

EXPLAIN IN ONE BREATH

JWS signs or authenticates content for integrity; JWE encrypts content for confidentiality.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
JWS -> recipient can read claims, verifies signature/MAC
JWE -> encrypted content, recipient needs decryption capability
JWT -> claims format used with these supported representations
```

Why it works: Encryption and signature policies can be combined in a designed protocol. An encrypted token still needs trusted keys and claim validation. Use reviewed libraries and identity-provider contracts rather than custom token formats.

⚠ **Interview trap:** HTTPS and token encryption address different exposure paths.

FOLLOW-UP QUESTION — ANSWERED

Why can a signed token still reveal personal data?

JWS protects integrity, not confidentiality: anyone holding the token can base64url-decode readable header/payload claims.

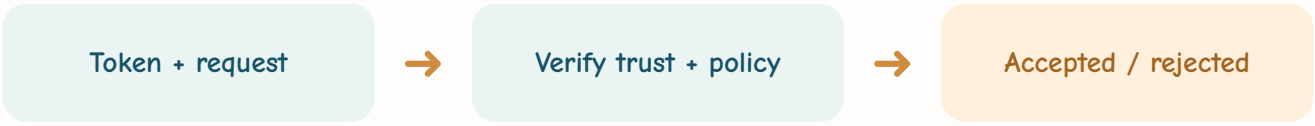
INTERVIEW QUESTION 057

How do symmetric and asymmetric JWT signatures differ?

EXPLAIN IN ONE BREATH

A symmetric MAC uses a shared secret; an asymmetric signature separates private signing from public verification keys.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
HS256: signing and verification share a secret
RS256/ES256: issuer signs privately, APIs verify publicly
```

Why it works: Asymmetric design lets multiple resource servers verify without receiving token-minting capability. Symmetric trust must tightly control every holder of the shared secret. Key length and allowed algorithm policy matter.

⚠ **Interview trap:** A public key is not a signing secret.

FOLLOW-UP QUESTION — ANSWERED

What would a compromised API learn under each design?

With HMAC, a verifier possessing the shared key could also mint tokens; asymmetric verification exposes only a public key.

INTERVIEW QUESTION 058

What do iss, sub, aud, exp and nbf mean?

EXPLAIN IN ONE BREATH

They identify issuer, subject, intended recipient, expiry, and not-before time; validate the claims relevant to the API's trust contract.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
iss=https://id.example/issuer
sub=user-123; aud=orders-api
exp=<expiry>; nbf=<earliest valid time>
```

Why it works: sub is meaningful within its issuer context and is not automatically an email or application database id. The intended audience must include this API. Clock policy should allow only a justified small tolerance.

⚠ **Interview trap:** Subject alone can collide across independent issuers.

FOLLOW-UP QUESTION — ANSWERED

How would you form a stable federated identity key?

Treat (issuer, subject) as the stable federated identity key; sub alone can collide across issuers.

INTERVIEW QUESTION 059

What is algorithm confusion and how is it avoided?

EXPLAIN IN ONE BREATH

It is accepting a token under an unintended algorithm/key interpretation; constrain trusted algorithms and correct key types through supported decoder configuration.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Expected policy: issuer signs with approved asymmetric algorithm
Reject unsupported alg, invalid signature and untrusted key material
Never choose arbitrary validation behavior solely from token input.
```

Why it works: The token's header is untrusted until verification. A vetted library still needs a secure decoder and key source. Do not accept unsigned tokens for protected API authentication.

⚠ **Interview trap:** The alg field is not permission to use any advertised algorithm.

FOLLOW-UP QUESTION — ANSWERED

How would a negative test exercise an unexpected algorithm?

Send a token signed with an unsupported or unexpected algorithm, and assert the configured decoder rejects it.

INTERVIEW QUESTION 060

How do kid and JWKS help key rotation?

EXPLAIN IN ONE BREATH

kid identifies a candidate key in a trusted key set; JWKS publishes supported verification keys for the issuer.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Issuer signs with new key kid=B
Trusted JWKS exposes A and B during overlap
Resource server resolves B and validates signature
```

Why it works: Key id is a lookup hint, not proof of trust. Do not fetch an arbitrary attacker-provided URL from the token header. Plan cache refresh, availability, overlap and retirement according to token lifetimes.

⚠ **Interview trap:** Reusing kid for different key material can confuse caches.

FOLLOW-UP QUESTION — ANSWERED

What happens to tokens signed by a retired key?

Retain old public verification keys only for the defined outstanding-token lifetime; after retirement, old signed tokens must fail.

11

JWT decoding, validation and authority mapping

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

JWT candidate

Issuer / time / audience

Map authorities

061 · What does JwtDecoder do?

062 · What does issuer-uri configuration provide?

063 · How do you add an audience validator safely?

064 · How are OAuth scopes mapped to Spring authorities by default?

INTERVIEW QUESTION 061

What does JwtDecoder do?

EXPLAIN IN ONE BREATH

It decodes and verifies a JWT under configured cryptographic and claim-validation policy, returning trusted claims or failure.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Jwt verified = decoder.decode(accessToken);
String subject = verified.getSubject();
// Do not substitute an unverified Base64 parser.
```

Why it works: A resource server uses the decoder during bearer authentication. Customize it for issuer, audience, supported algorithms and other contractual constraints. Token parsing success alone is insufficient.

⚠ **Interview trap:** JwtDecoder does not establish object ownership automatically.

FOLLOW-UP QUESTION — ANSWERED

Which failure should become an authentication response?

Signature/claim failures belong to authentication error handling (normally 401), not a successful anonymous fallback.

INTERVIEW QUESTION 062

What does issuer-uri configuration provide?

EXPLAIN IN ONE BREATH

It supplies a trusted issuer for supported metadata/key discovery and issuer-based validation, with behavior depending on decoder configuration and version.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://id.example/issuer
// Resource-server chain and relevant dependencies are also required.
```

Why it works: Do not assume this automatically enforces a custom application audience. Discovery and key access can introduce startup or runtime dependencies; check the selected release's behavior. Explicit key-set configuration can change discovery coupling.

⚠ **Interview trap:** The request must not choose an arbitrary issuer endpoint.

FOLLOW-UP QUESTION — ANSWERED

How would a known JWKS URI affect discovery?

A directly configured trusted jwk-set-uri may avoid discovery startup dependency; issuer validation must still be configured explicitly.

INTERVIEW QUESTION 063

How do you add an audience validator safely?

EXPLAIN IN ONE BREATH

Compose audience validation with default time and issuer checks rather than replacing them with a single custom predicate.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
OAuth2TokenValidator<Jwt> checks = new DelegatingOAuth2TokenValidator<>(
JwtValidators.createDefaultWithIssuer(issuer), audienceValidator);
decoder.setJwtValidator(checks);
```

Why it works: The custom audience validator should require the intended API recipient and return a structured validation failure. Keeping default checks prevents accepting an expired or wrong-issuer token merely because audience matched.

⚠ **Interview trap:** setJwtValidator replaces the selected validator; composition is deliberate.

FOLLOW-UP QUESTION — ANSWERED

How would tests prove both expiry and audience rejection?

Test expired tokens and wrong audiences separately with a real decoder; then assert the combination of validators rejects both.

INTERVIEW QUESTION 064

How are OAuth scopes mapped to Spring authorities by default?

EXPLAIN IN ONE BREATH

The supported resource-server converter commonly maps scope or scp values into authorities prefixed with SCOPE_.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Token scope: orders.read orders.write
Authorities: SCOPE_orders.read, SCOPE_orders.write
Rule: hasAuthority("SCOPE_orders.read")
```

Why it works: Roles, groups and custom permissions require a trusted conversion policy when they use another claim shape. Distinguish client-requested scope from issued approved scope. Verify exact provider claim types.

⚠ **Interview trap:** hasRole("orders.read") does not check the default scope authority.

FOLLOW-UP QUESTION — ANSWERED

How would a list-valued roles claim be mapped?

Convert only approved role strings to ROLE_ authorities using a trusted claim/issuer; handle absent and malformed list types safely.

INTERVIEW QUESTION 065

How can JwtAuthenticationConverter customize identity and authorities?

EXPLAIN IN ONE BREATH

It selects principal naming and granted-authority conversion from verified claims under the application's policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
JwtGrantedAuthoritiesConverter rights = new JwtGrantedAuthoritiesConverter();
rights.setAuthoritiesClaimName("permissions");
rights.setAuthorityPrefix("");
JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
converter.setJwtGrantedAuthoritiesConverter(rights);
```

Why it works: Wire the converter into the resource-server JWT DSL. This example assumes a supported permissions claim shape from a trusted issuer. Mapping a different claim can stop default scopes from being included unless you compose them.

⚠ **Interview trap:** Claim conversion must not create arbitrary privileges from untrusted inputs.

FOLLOW-UP QUESTION — ANSWERED

How would you preserve both scopes and roles?

Keep the default scope extractor, then add allowlisted roles from the verified issuer, not replace scopes accidentally.

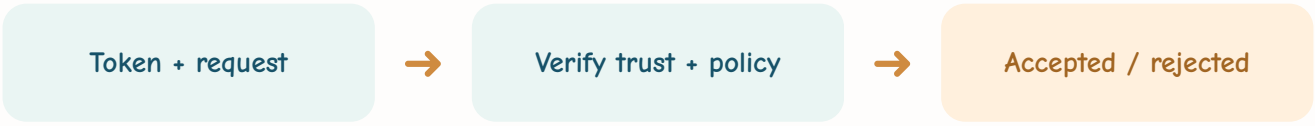
INTERVIEW QUESTION 066

How should expired, invalid and malformed tokens be handled?

EXPLAIN IN ONE BREATH

Reject them through the bearer authentication failure contract without falling back to an invented authenticated identity.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Malformed token -> reject
Bad signature or issuer -> reject
Expired beyond allowed skew -> reject
Valid token, insufficient scope -> authorization denial
```

Why it works: Keep safe diagnostic categories without recording complete credentials. Anonymous optional routes need a deliberate behavior when an invalid credential is supplied; permitAll does not guarantee a supplied bad bearer token is ignored.

⚠ **Interview trap:** Never trust claims after decoder failure.

FOLLOW-UP QUESTION — ANSWERED

How would you distinguish authentication failure from permission denial in metrics?

Count invalid/expired/malformed credentials as authentication failures and missing rights as authorization denials without logging tokens.

12

OAuth 2.0 flows and client responsibilities

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

OAuth client | Grant / PKCE consent | Access token

067 · What are OAuth's main roles?

068 · How does authorization code with PKCE work?

069 · What is client credentials grant for?

070 · How do public and confidential clients differ?

INTERVIEW QUESTION 067

What are OAuth's main roles?

EXPLAIN IN ONE BREATH

The resource owner grants access, the client obtains a token, the authorization server issues it, and the resource server enforces access to protected resources.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
User -> approves access at authorization server
Client -> obtains access token
Client -> calls API resource server with that token
```

Why it works: The roles can run in separate services. The resource server does not become a token issuer just because it validates JWTs. Identify who owns user authentication, client registration and consent.

⚠ **Interview trap:** OAuth is not simply 'JWT login'.

FOLLOW-UP QUESTION — ANSWERED

Which role is your Spring API implementing?

A protected Spring API typically acts as the resource server; the OAuth authorization server issues access tokens.

INTERVIEW QUESTION 068

How does authorization code with PKCE work?

EXPLAIN IN ONE BREATH

The client starts authorization with a challenge and later exchanges the returned code with its matching verifier.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
1. Authorization request: challenge + state + redirect URI
2. Callback: authorization code
3. Token exchange: code + verifier
```

Why it works: Validate state and redirect handling, and OIDC nonce where applicable. PKCE reduces intercepted-code abuse but does not replace every client or browser control. Each code and verifier needs correct lifetime and single-operation ownership.

⚠ **Interview trap:** A public client cannot keep a browser-delivered static secret confidential.

FOLLOW-UP QUESTION — ANSWERED

How do you bind a callback to its initiating session?

Use state to bind the callback to the initiating browser flow, then validate registered redirect URI and PKCE proof.

INTERVIEW QUESTION 069

What is client credentials grant for?

EXPLAIN IN ONE BREATH

It authorizes a client application acting for itself, not a human login delegation flow.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Batch service -> token endpoint using trusted client authentication
Issued access token -> approved service scopes
Batch service -> resource API
```

Why it works: The API's permissions should reflect service identity and its allowed capabilities. There is no automatic current-user principal just because the token is valid. Keep secrets or key credentials out of code and browser apps.

⚠ **Interview trap:** Do not use this grant as a user sign-in replacement.

FOLLOW-UP QUESTION — ANSWERED

How would tenant access be constrained for a service principal?

Use a service identity with tenant-specific authority or audience; do not infer every customer's rights from a client_credentials token.

INTERVIEW QUESTION 070

How do public and confidential clients differ?

EXPLAIN IN ONE BREATH

A confidential client can protect an authentication credential in its deployment; a public client cannot rely on a delivered static secret remaining confidential.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Backend web service -> potentially confidential client
Browser SPA/native app -> typically public client
PKCE -> protects code exchange for applicable flows
```

Why it works: Choose registered authentication methods from the actual execution environment. Obfuscation in JavaScript or a mobile binary does not create a trustworthy shared secret. Server-side key authentication may fit confidential clients.

⚠ **Interview trap:** Client id is an identifier, not a secret.

FOLLOW-UP QUESTION — ANSWERED

Why must redirect URIs still be constrained?

Registered redirect URIs prevent authorization codes/tokens from being sent to an attacker-controlled callback.

INTERVIEW QUESTION 071

Why avoid implicit and password-grant designs for new applications?

EXPLAIN IN ONE BREATH

They create avoidable token/credential exposure and responsibility problems; prefer modern supported flows such as code with PKCE or client credentials where appropriate.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
User-facing client -> authorization code + PKCE
Service-to-service -> client credentials with narrow scopes
Legacy password forwarding -> migration problem, not default design
```

Why it works: Use current provider and protocol guidance for the intended client. Do not make an unrelated app collect identity-provider passwords merely to obtain a token. Legacy interoperability needs a specific plan.

⚠ **Interview trap:** A convenient one-step token endpoint is not a sound client boundary.

FOLLOW-UP QUESTION — ANSWERED

How would a legacy password flow be migrated?

Move to authorization code + PKCE, stop password collection by the client, and migrate/rotate credentials and consent deliberately.

INTERVIEW QUESTION 072

How should scopes and consent be designed?

EXPLAIN IN ONE BREATH

Scopes represent approved delegated capabilities; minimize them and separate them from domain ownership and client registration policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Requested: orders.read orders.write
Granted: orders.read
API accepts only the issued scope and still checks order ownership.
```

Why it works: The authorization server decides approved grant policy. Scope names should have stable meaning rather than act as arbitrary role strings. User consent may differ from machine-client approval.

⚠ **Interview trap:** A token can request no rights merely by inventing a scope string.

FOLLOW-UP QUESTION — ANSWERED

Who limits a client's maximum permitted scopes?

The authorization server enforces the registered client's allowed scopes and user consent/authorization policy.

13

OIDC login and federated identity

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

OIDC callback

state + nonce check

Identity session

073 · What does OpenID Connect add to OAuth?

074 · How do state and nonce differ?

075 · How does oauth2Login configure browser authentication?

076 · What is an OAuth2AuthorizedClient?

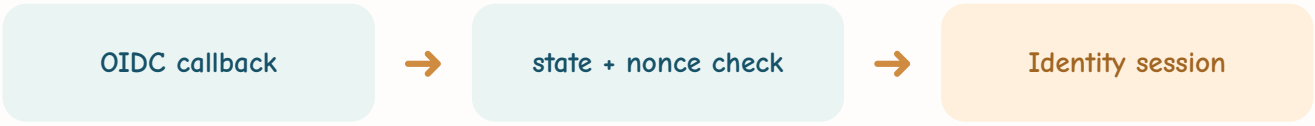
INTERVIEW QUESTION 073

What does OpenID Connect add to OAuth?

EXPLAIN IN ONE BREATH

It adds an identity layer with ID tokens, standardized claims and supported authentication/discovery behavior.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
OAuth access token -> protected resource access
OIDC ID token -> client's authentication result
UserInfo -> provider-defined identity claims through supported access
```

Why it works: Use the provider's OIDC validation and login integration. JWT formatting alone does not turn a homemade token into OIDC. Identity assurance and claims depend on the provider contract.

⚠ **Interview trap:** An ID token is not automatically a valid API access token.

FOLLOW-UP QUESTION — ANSWERED

Which audience should an ID token target?

The ID token's audience is the OAuth/OIDC client application (client_id), not an arbitrary downstream resource server.

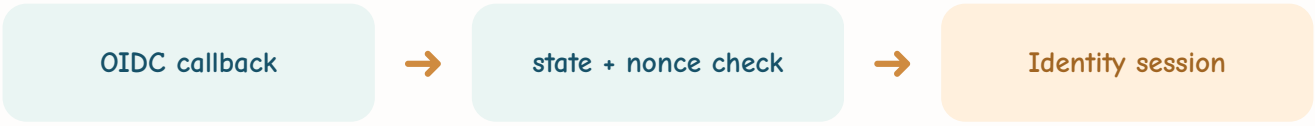
INTERVIEW QUESTION 074

How do state and nonce differ?

EXPLAIN IN ONE BREATH

State binds the authorization callback to the initiating flow; OIDC nonce binds an ID token to the expected authentication request.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authorization request -> random state and applicable nonce
Callback -> validate matching state
ID token -> validate expected nonce through supported flow
```

Why it works: Keep values unpredictable and tied to the correct operation. Their exact storage and validation are managed by a supported client integration when configured correctly. PKCE binds a separate code-exchange proof.

⚠ **Interview trap:** State, nonce and PKCE are not interchangeable.

FOLLOW-UP QUESTION — ANSWERED

Which attack remains if redirect validation is missing?

Even with valid state/nonce, a loose redirect URI or open redirect can leak authorization results to an attacker.

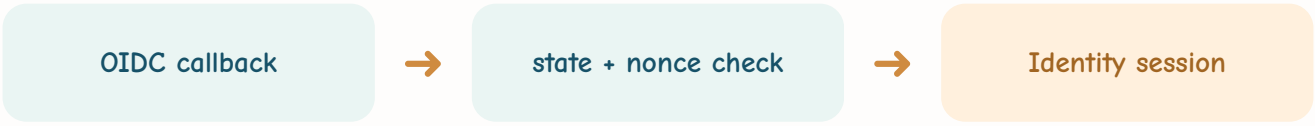
INTERVIEW QUESTION 075

How does oauth2Login configure browser authentication?

EXPLAIN IN ONE BREATH

It integrates the application as an OAuth/OIDC client handling authorization redirects, callbacks and authenticated identity.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
http.authorizeHttpRequests(a -> a.anyRequest().authenticated())
.oauth2Login(Customizer.withDefaults());
// Configure client registration/provider settings externally.
```

Why it works: The application needs approved registration, redirect URI and credential policy. Browser login commonly uses session state. This is different from oauth2ResourceServer accepting bearer tokens at an API.

⚠ **Interview trap:** Enabling login does not automatically secure an API's token audience.

FOLLOW-UP QUESTION — ANSWERED

Can one app use separate login and API chains?

Yes: use ordered browser OAuth login and a separate narrow bearer-resource-server API chain with a deny-by-default fallback.

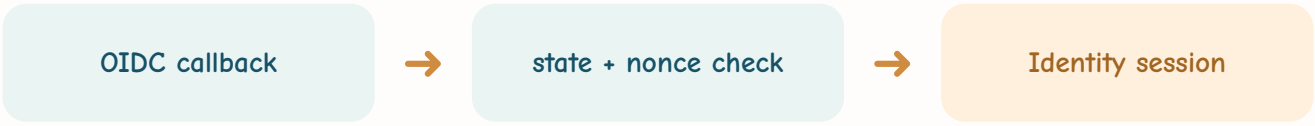
INTERVIEW QUESTION 076

What is an OAuth2AuthorizedClient?

EXPLAIN IN ONE BREATH

It represents a registered client associated with an authenticated principal and authorized tokens for calling protected resources.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Principal + ClientRegistration
-> OAuth2AuthorizedClient
-> approved access token, optional refresh token
```

Why it works: A manager/repository/service controls storage and renewal according to the chosen integration. It is not the same object as an end-user account. Protect retained credentials and define their lifecycle.

⚠ **Interview trap:** A user's login session and outbound token can expire independently.

FOLLOW-UP QUESTION — ANSWERED

How would a background call obtain client credentials safely?

A background service uses a supported OAuth2AuthorizedClientManager/client-credentials grant with protected server credentials.

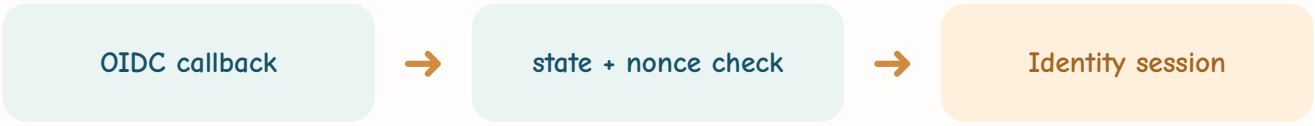
INTERVIEW QUESTION 077

How should external claims become local permissions?

EXPLAIN IN ONE BREATH

Map verified provider identity to local account and permission policy through an explicit trusted translation.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Identity key: (issuer, subject)
Local link -> application user id
Verified group/claim -> approved local permission mapping
```

Why it works: Email can change and may not be a stable unique identity unless the contract guarantees it. Avoid granting administrator rights from an arbitrary claim without provider and client policy. Account linking needs authenticated proof.

⚠ **Interview trap:** Matching email alone can create account-takeover risk in federation.

FOLLOW-UP QUESTION — ANSWERED

How would two issuers with the same sub be distinguished?

Map (trusted issuer, subject) to local identity and obtain application privileges from an approved authority, not raw display names.

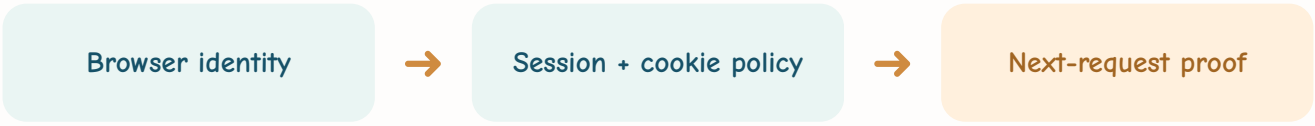
INTERVIEW QUESTION 078

What does single logout require beyond local session clearing?

EXPLAIN IN ONE BREATH

Coordinate the local session/token lifecycle with the identity provider's supported logout protocol and the desired application scope.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Local logout -> application session removed
Provider logout -> supported OIDC/SAML mechanism
Other apps -> may retain independent sessions unless coordinated
```

Why it works: Define what 'log out everywhere' means, including refresh tokens, remembered sessions, and existing access-token lifetime. Redirect logout and back-channel capabilities are provider/version dependent.

⚠ **Interview trap:** Clearing one cookie does not revoke every issued token.

FOLLOW-UP QUESTION — ANSWERED

How would you test logout across two applications?

Use two client apps sharing an IdP, sign out at one, and verify the other app's local/federated session behavior against the contract.

14

Resource-server tokens, refresh and revocation

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Access or refresh

Introspection / renewal

Revocation policy

079 · How do JWT and opaque access tokens differ for resource servers?

080 · What does opaqueToken resource-server configuration do?

081 · How do access and refresh tokens differ?

082 · What is refresh-token rotation and reuse detection?

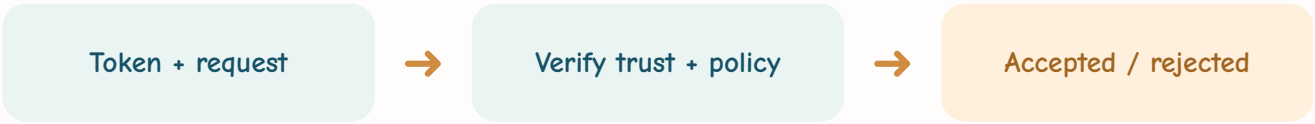
INTERVIEW QUESTION 079

How do JWT and opaque access tokens differ for resource servers?

EXPLAIN IN ONE BREATH

JWT can be validated locally under trusted keys and claims; opaque tokens commonly require authorization-server introspection or another authoritative lookup.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
JWT -> local signature/claims validation
Opaque -> introspection result and active-status policy
// Both can be bearer access tokens under OAuth.
```

Why it works: Local validation reduces request-time coupling but delays some revocation knowledge. Introspection adds latency and availability requirements; caching changes freshness. Choose from actual risk and provider behavior.

⚠ **Interview trap:** Opaque does not mean 'less secure', and JWT does not mean 'always faster enough'.

FOLLOW-UP QUESTION — ANSWERED

Which design supports immediate centralized active-status checks?

Opaque-token introspection checks active state centrally when online, subject to caches and introspection availability.

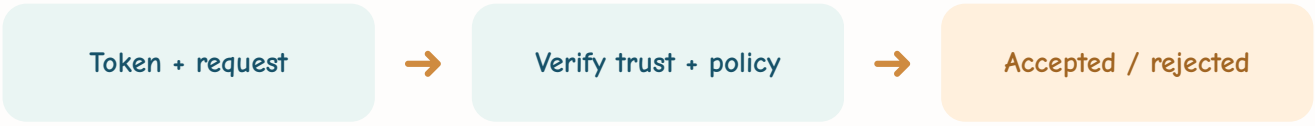
INTERVIEW QUESTION 080

What does opaqueToken resource-server configuration do?

EXPLAIN IN ONE BREATH

It enables bearer authentication by introspecting opaque tokens through the configured trusted service.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
http.oauth2ResourceServer(o -> o.opaqueToken(Customizer.withDefaults()));
# Configure introspection URI and protected client credentials.
```

Why it works: The returned active status and authorities follow provider/converter policy. Configure deadline, credential protection and availability handling. Do not accept tokens merely because introspection failed or timed out.

⚠ **Interview trap:** Caching positive responses can delay revocation effects.

FOLLOW-UP QUESTION — ANSWERED

What status should an unavailable identity dependency produce?

Fail authentication safely and surface dependency unavailability through the API policy; never permit anonymous access because introspection failed.

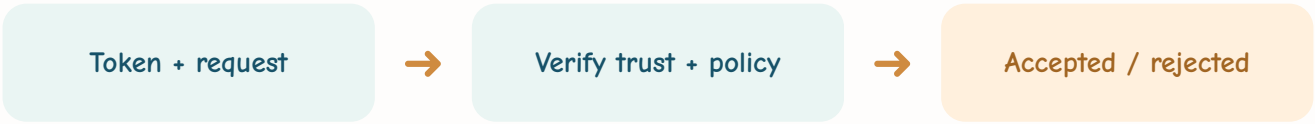
INTERVIEW QUESTION 081

How do access and refresh tokens differ?

EXPLAIN IN ONE BREATH

Access tokens authorize resource calls; refresh tokens are presented to the authorization server to obtain renewed access tokens.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
API request -> access token
Token endpoint renewal -> refresh token
API must not treat refresh token as a resource credential.
```

Why it works: Not every flow issues a refresh token. Store refresh credentials according to client architecture, and enforce provider rotation and client binding. Their longer useful lifetime makes theft especially consequential.

⚠ **Interview trap:** A refresh token is not a remember-me cookie.

FOLLOW-UP QUESTION — ANSWERED

How does a public client store tokens without a static secret?

A public browser/native client cannot keep a static secret; use PKCE and an appropriate protected refresh-token/storage strategy.

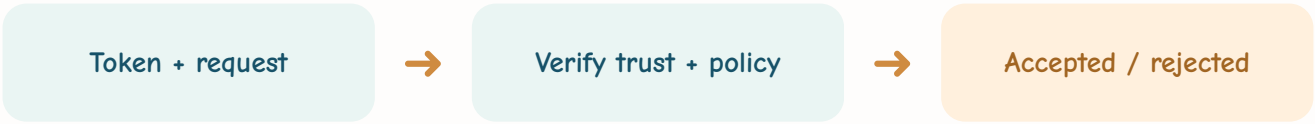
INTERVIEW QUESTION 082

What is refresh-token rotation and reuse detection?

EXPLAIN IN ONE BREATH

Rotation replaces a refresh token during renewal; reuse detection identifies later attempts with an already-replaced token and applies the provider's compromise policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Refresh A -> issue B, retire A
A used again -> detect reuse and apply token-family policy
```

Why it works: Concurrent renewals and retries need a clear client coordination and provider policy. The resource server typically does not implement refresh-token issuance. Define what becomes invalid after suspected theft.

⚠ **Interview trap:** Rotation alone without authoritative tracking does not detect every reuse.

FOLLOW-UP QUESTION — ANSWERED

How would simultaneous browser tabs coordinate renewal?

Serialize or coordinate renewal per session, rotate tokens atomically, and handle reuse detection and overlapping tab refreshes.

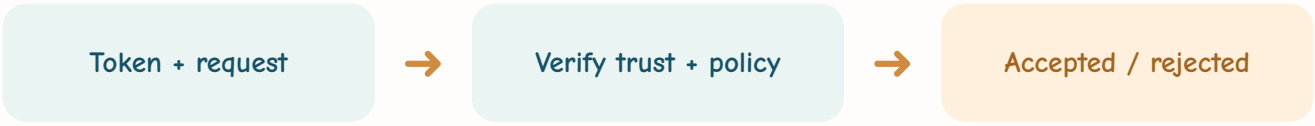
INTERVIEW QUESTION 083

Why can logout leave a JWT usable until expiry?

EXPLAIN IN ONE BREATH

A self-contained token validated only from signature and claims has no automatic knowledge of the local logout event.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Logout at 12:00
JWT exp at 12:10
Local cryptographic validation may accept until expiry unless another policy intervenes.
```

Why it works: Short lifetime, denylisting, token-version checks or introspection can improve revocation according to requirements. Deleting the client's copy does not invalidate a stolen one. Include operational cost and outages in the design.

⚠ **Interview trap:** Signing-key rotation is not a precise one-user logout action.

FOLLOW-UP QUESTION — ANSWERED

What revocation latency is acceptable for administrators?

Choose an explicit short revocation bound for admins (possibly near-immediate); implement central checks or denylist if required.

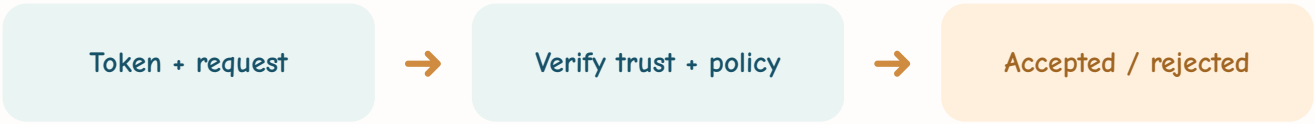
INTERVIEW QUESTION 084

What is a token-replay threat and what controls help?

EXPLAIN IN ONE BREATH

A stolen bearer token can generally be replayed by its holder; use protected transport/storage, short scope/lifetime and supported sender-constraining where required.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Bearer secret stolen -> attacker sends same Authorization header
Possible controls -> TLS, reduced audience/scope/lifetime,
supported mTLS/DPoP protocol integration
```

Why it works: Sender constraints require end-to-end client, issuer and resource-server support; adding a random claim does not implement them. Avoid logging credentials or placing them in URLs.

⚠ **Interview trap:** JWT signature verifies issuer, not possession by the original browser.

FOLLOW-UP QUESTION — ANSWERED

When would a sender-constrained token be worth the complexity?

Sender-constrained tokens help high-risk APIs where stolen bearer-token replay is serious enough to justify mTLS/DPoP complexity.

15

Authorization servers and signing-key lifecycle

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Registered client

Signing key lifecycle

Token trust

085 · How does an authorization server differ from a resource server?

086 · What does a RegisteredClient describe?

087 · What should production signing-key management include?

088 · How should redirect URIs be validated?

INTERVIEW QUESTION 085

How does an authorization server differ from a resource server?

EXPLAIN IN ONE BREATH

The authorization server authenticates relevant actors and issues approved tokens; the resource server validates credentials and enforces resource permissions.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authorization server -> authorization/token endpoints
Resource server -> /orders and other protected business APIs
Client -> participates in the chosen grant
```

Why it works: A Spring resource-server starter does not turn an API into a complete identity provider. Use supported authorization-server software or a trusted external provider with reviewed operations.

⚠ **Interview trap:** Homemade password-to-JWT controllers omit many protocol responsibilities.

FOLLOW-UP QUESTION — ANSWERED

Who owns consent and registered redirect URIs?

The authorization server owns registered clients, redirect URIs and consent; the resource server enforces issued-token access rules.

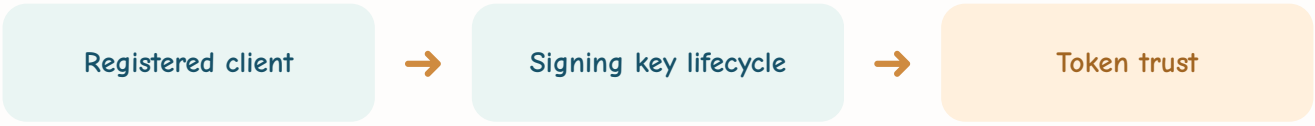
INTERVIEW QUESTION 086

What does a RegisteredClient describe?

EXPLAIN IN ONE BREATH

It stores a client's identity and permitted authentication methods, grant types, redirects, scopes and related settings in supported server models.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Client id -> identifier
Authentication method -> secret/key/public policy
Redirects + grants + scopes -> approved client capabilities
```

Why it works: Persist production registration under a controlled lifecycle and audit changes. Public clients should not be configured as though they can protect a delivered secret. Registration is distinct from user-account storage.

⚠ **Interview trap:** An in-memory development repository is not a durable production registry.

FOLLOW-UP QUESTION — ANSWERED

How would you revoke one client's authorization?

Revoke related authorizations/grants and refresh tokens using the server's supported persistence and revocation mechanism.

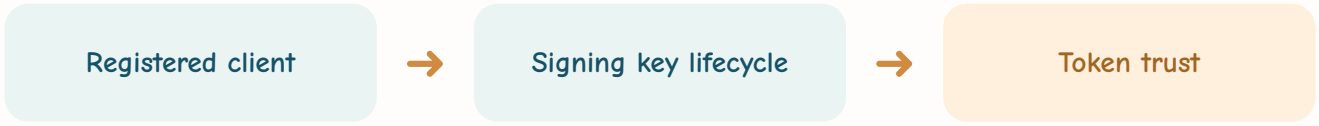
INTERVIEW QUESTION 087

What should production signing-key management include?

EXPLAIN IN ONE BREATH

Protect private keys, limit signer access, rotate with overlap, publish trusted verification keys, and plan compromise recovery.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Active signing key -> private protected material
JWKS -> approved public verification keys
Rotation -> publish new, switch signer, retire old after policy window
```

Why it works: Retention must consider still-valid tokens and emergency revocation. Persist keys across expected restarts rather than generating a new ephemeral signer accidentally. Hardware or managed key services can fit stronger requirements.

⚠ **Interview trap:** Publishing a public JWKS must not leak private parameters.

FOLLOW-UP QUESTION — ANSWERED

How would you recover from a private-key compromise?

Emergency-rotate the signing key, stop issuing from it, revoke/shorten affected tokens, publish trusted replacements and audit exposure.

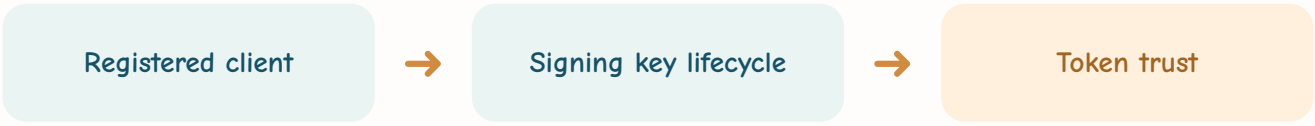
INTERVIEW QUESTION 088

How should redirect URIs be validated?

EXPLAIN IN ONE BREATH

Register and enforce approved redirect targets through supported protocol rules, avoiding broad attacker-controlled destinations.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Registered: https://app.example/login/oauth2/code/company
Unexpected: https://evil.example/callback -> reject
Loopback native-client cases -> use the supported protocol policy.
```

Why it works: Redirect validation, state and PKCE address different requirements. Open redirects in your application can still create credential or code exposure. Do not assume wildcard matching is safe because a domain looks familiar.

⚠ **Interview trap:** A client-supplied returnUrl is not automatically trusted.

FOLLOW-UP QUESTION — ANSWERED

How would post-login navigation avoid open redirects?

Match exact registered URIs; post-login navigation should use local allowlisted paths, not arbitrary return URLs.

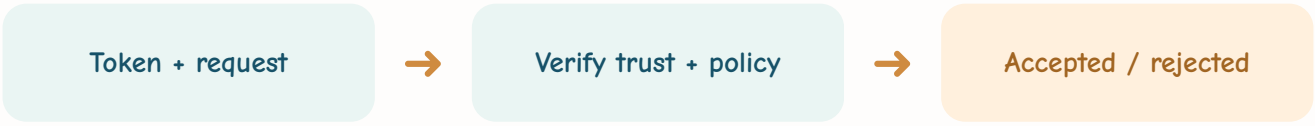
INTERVIEW QUESTION 089

Why should custom grants or token claims be added cautiously?

EXPLAIN IN ONE BREATH

They expand the trust contract and interoperability surface, so each needs clear issuer authority, validation and lifecycle rules.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Claim tenant_id -> authoritative issuer policy
Claim permissions -> defined approved mapping
Custom grant -> reviewed client authentication and replay controls
```

Why it works: Do not copy arbitrary request attributes into approved credentials. Minimize personal data and claim size, and version meaning deliberately. A resource server must agree on the interpretation.

⚠ **Interview trap:** More claims do not mean stronger authorization.

FOLLOW-UP QUESTION — ANSWERED

Which claim should stay in an authoritative database instead?

Keep frequently changing, sensitive entitlement and current account status in an authoritative service/database rather than stale long-lived tokens.

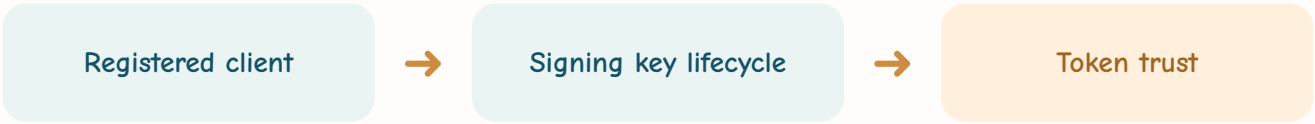
INTERVIEW QUESTION 090

How do MFA and step-up authentication fit the design?

EXPLAIN IN ONE BREATH

They establish additional assurance through a supported factor and authentication flow, with sensitive operations requiring trusted evidence of the necessary assurance.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Password login -> baseline assurance
Approve high-value action -> require supported stronger/recent authentication
Resource policy -> check trusted assurance claims/state as specified
```

Why it works: Do not infer MFA solely from a local role or token signature. Provider support, factor recovery and freshness policy matter. Legacy and modern framework integrations vary by version.

⚠ **Interview trap:** A second page asking the same password is not an independent factor.

FOLLOW-UP QUESTION — ANSWERED

How would account recovery avoid bypassing MFA?

Offer secure recovery codes and verified reset flows with equivalent assurance; do not let a weak recovery channel bypass MFA.

16

Multi-tenant and service-to-service boundaries

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Tenant + workload

Trusted scope + owner

Data isolation

091 · How should tenant identity be resolved?

092 · How should multiple JWT issuers be trusted?

093 · How should machine-client permissions differ from user permissions?

094 · Should inbound access tokens be forwarded to every downstream service?

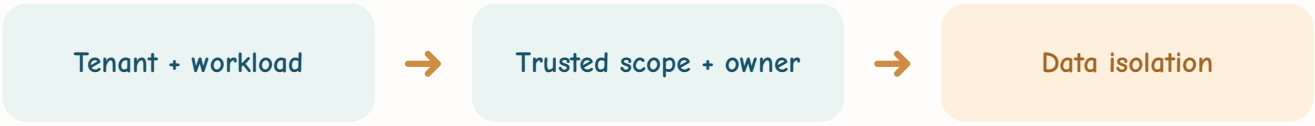
INTERVIEW QUESTION 091

How should tenant identity be resolved?

EXPLAIN IN ONE BREATH

Resolve it from a verified identity or an independently authorized selection, then enforce its relation to the requested data.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Verified issuer + subject -> approved tenant memberships
Requested tenant -> confirm membership
Repository query -> tenant predicate + object id
```

Why it works: A header can express a selection but cannot alone prove membership. Service identities may require a different tenant policy from end users. Carry only trusted normalized context downstream.

⚠ **Interview trap:** The browser choosing tenant_id does not create authorization.

FOLLOW-UP QUESTION — ANSWERED

What if a user belongs to several tenants?

Require an explicit selected tenant among authorized memberships, and enforce it on every data query and cache key.

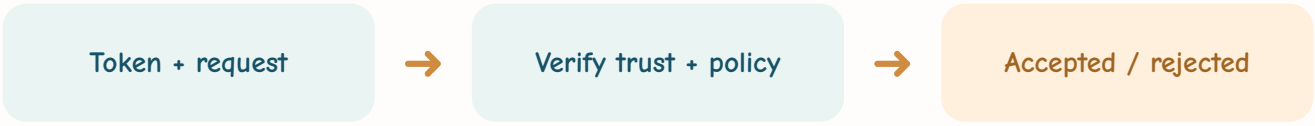
INTERVIEW QUESTION 092

How should multiple JWT issuers be trusted?

EXPLAIN IN ONE BREATH

Use a finite approved issuer/decoder policy and reject untrusted issuer values instead of discovering arbitrary URLs from incoming tokens.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Allowed issuers: company-A, company-B
Incoming iss -> choose only an approved authentication manager
Each manager -> its own keys, audience and claim policy
```

Why it works: Issuer resolution commonly inspects unverified data only to choose a trusted policy; the selected manager must then verify it. Limit network targets and authority mappings independently.

⚠ **Interview trap:** Arbitrary discovery can create SSRF and trust escalation.

FOLLOW-UP QUESTION — ANSWERED

How would the same subject string differ across issuers?

Treat identity as (issuer, subject), and configure allowlisted issuers/algorithms/keys; a repeated sub across issuers is not one person.

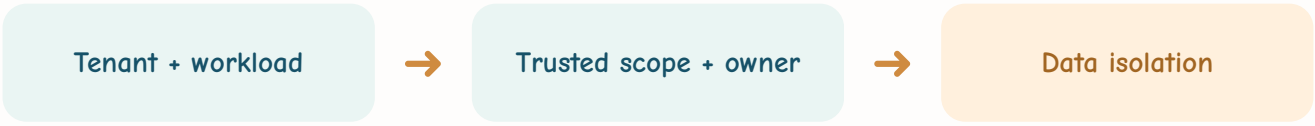
INTERVIEW QUESTION 093

How should machine-client permissions differ from user permissions?

EXPLAIN IN ONE BREATH

Model the service principal's approved capabilities explicitly rather than pretending every client-credentials token represents a human.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Service billing-worker -> scope orders.settle
User account -> own-order read/modify policy
Audit -> record actual actor type and operation identity
```

Why it works: A service may legitimately act across users but should have narrow audience and scope. Avoid granting universal administrator capability simply because it is internal. Track delegated versus service-owned actions correctly.

⚠ **Interview trap:** Internal network location is not sufficient identity.

FOLLOW-UP QUESTION — ANSWERED

When would on-behalf-of delegation be required?

Machine roles should be tightly scoped; delegation is required when the downstream must act on behalf of a real user.

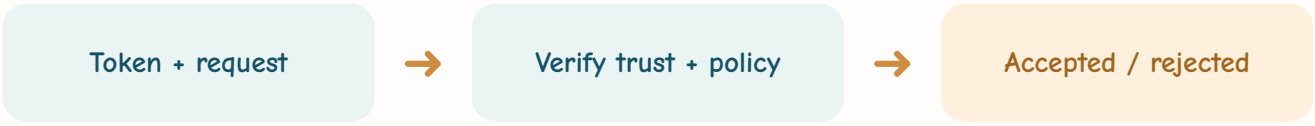
INTERVIEW QUESTION 094

Should inbound access tokens be forwarded to every downstream service?

EXPLAIN IN ONE BREATH

Forward only when audience, delegation and trust contracts allow it; otherwise obtain an appropriate downstream credential.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Inbound token audience=orders-api
Downstream audience=billing-api
Blind relay -> billing may reject, or unsafe validation may overtrust it.
```

Why it works: Use approved token exchange, client credentials, or authorized-client integration as appropriate. Keep downstream scope narrow and avoid accidental privilege amplification. Access-token relay is not automatic universal delegation.

⚠ **Interview trap:** A token valid at one API need not be valid at another.

FOLLOW-UP QUESTION — ANSWERED

Who should be audited for a delegated payment?

Audit both service client and delegated end-user identity, plus transaction/tenant correlation IDs and decision outcome.

INTERVIEW QUESTION 095

How can cached permissions or data leak authorization?

EXPLAIN IN ONE BREATH

A cache hit can bypass method execution or reuse a result across a missing user/tenant/policy dimension.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Outer service: authorize caller on every request
Inner cache: tenant-safe authorization-independent data
Cache key: all result-affecting dimensions
```

Why it works: Do not place the only ownership check inside a @Cacheable method that is skipped on hits. Decide invalidation for membership or permission changes. Authority caching needs a defined revocation lag.

⚠ **Interview trap:** A structurally immutable DTO can still contain another tenant's data.

FOLLOW-UP QUESTION — ANSWERED

How would a hit-path denial be tested?

Test a warmed cache entry for tenant A requested by tenant B and assert denial/no data leakage on a cache hit.

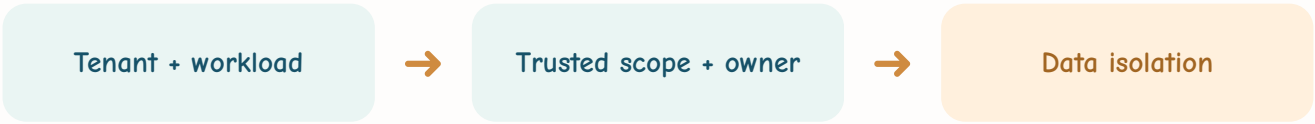
INTERVIEW QUESTION 096

How should database row rules complement application security?

EXPLAIN IN ONE BREATH

Use authoritative tenant-scoped queries and, where required, database isolation/row policies as defense in depth.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
UPDATE orders SET ...
WHERE id=:id AND tenant_id=:trustedTenant AND version=:version
```

Why it works: Check affected rows and distinguish permitted absence/conflict semantics safely. Application and database identity propagation must remain correct, especially with pooled connections. Direct alternate entry points should not bypass the invariant.

⚠ **Interview trap:** A controller filter does not protect batch or message paths automatically.

FOLLOW-UP QUESTION — ANSWERED

How would connection reuse affect per-session database tenant state?

Clear/reset session-level tenant variables on pooled connection return and always bind tenant constraints to the query/transaction.

17

Reactive and asynchronous security

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Async/reactive work | Context propagation | No leak / deny

097 · How does reactive SecurityWebFilterChain differ from servlet SecurityFilterChain?

098 · Where is authentication stored in a reactive chain?

099 · How does reactive method security work?

100 · Why should identity lookup not block a reactive event loop?

INTERVIEW QUESTION 097

How does reactive SecurityWebFilterChain differ from servlet SecurityFilterChain?

EXPLAIN IN ONE BREATH

It configures WebFlux security using reactive web filters and ServerHttpSecurity rather than servlet HttpSecurity filters.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@Bean
SecurityWebFilterChain api(ServerHttpSecurity http) {
return http.authorizeExchange(a -> a.anyExchange().authenticated())
.oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults())).build();
}
```

Why it works: Configure a ReactiveJwtDecoder and explicit credential/CSRF policy. Do not mix servlet request matchers or servlet filters into this chain. Reactive context and transport behavior require reactive integration.

⚠ **Interview trap:** Identical policy intent does not imply identical classes or callback signatures.

FOLLOW-UP QUESTION — ANSWERED

Which testing client fits WebFlux?

Use WebTestClient with reactive security test support and the actual SecurityWebFilterChain configuration.

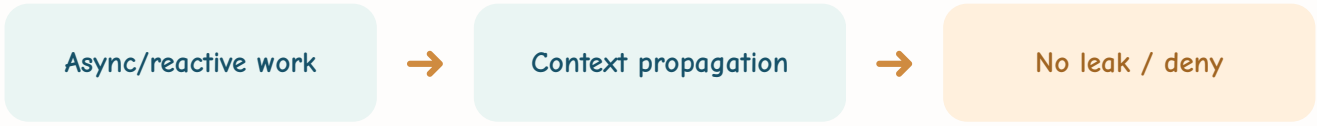
INTERVIEW QUESTION 098

Where is authentication stored in a reactive chain?

EXPLAIN IN ONE BREATH

Reactive security uses Reactor context through supported holders, not ordinary thread-local identity assumptions.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Mono<String> name = ReactiveSecurityContextHolder.getContext()
.map(c -> c.getAuthentication().getName());
```

Why it works: Execution can move among threads while the subscriber context carries supported state. An imperative SecurityContextHolder lookup on the current thread may not find the intended identity. Compose the lookup inside the chain.

⚠ **Interview trap:** Calling subscribe separately creates another context/ownership boundary.

FOLLOW-UP QUESTION — ANSWERED

How would context be propagated to a blocking bridge?

Carry security via Reactor Context or a deliberately managed context bridge; do not assume ThreadLocal follows reactive hops.

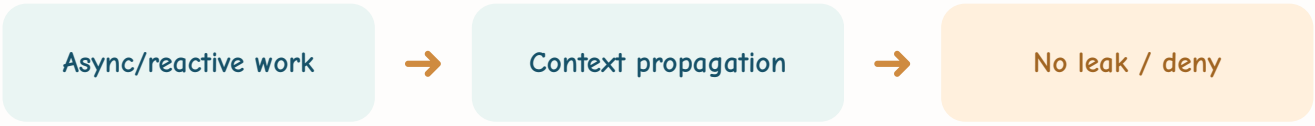
INTERVIEW QUESTION 099

How does reactive method security work?

EXPLAIN IN ONE BREATH

It applies supported authorization to reactive method results and context when enabled through the reactive method-security integration.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@EnableReactiveMethodSecurity
@Configuration
class ReactiveMethodConfig {}
@PreAuthorize("hasAuthority('SCOPE_orders.read')")
public Mono<OrderView> read(long id) { return repository.read(id); }
```

Why it works: Use compatible return types and policy APIs for the baseline. A publisher defers work, so subscription and context matter. Ensure the method does not perform an irreversible action eagerly before returning a publisher.

⚠ **Interview trap:** Mono.just(sideEffect()) runs the side effect immediately.

FOLLOW-UP QUESTION — ANSWERED

How would you test denial before deferred work occurs?

Use method-security tests around returned publishers; subscribe and assert denied calls do not execute deferred business work.

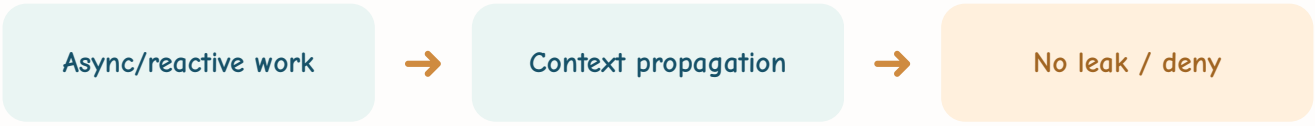
INTERVIEW QUESTION 100

Why should identity lookup not block a reactive event loop?

EXPLAIN IN ONE BREATH

Blocking directory/database calls can stall shared reactive processing; use an appropriate non-blocking integration or isolate known blocking work.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Mono<User> user = Mono.fromCallable(() -> blockingStore.load(id))
.subscribeOn(Schedulers.boundedElastic());
```

Why it works: This defers and isolates the blocking call, but still needs deadline and concurrency limits. An unlimited authentication backlog can become denial-of-service exposure. Offloading does not make the underlying store non-blocking.

⚠ **Interview trap:** Returning Mono after already performing the call is not isolation.

FOLLOW-UP QUESTION — ANSWERED

How would identity-store saturation be measured?

Measure event-loop blocked time, identity-store queue/backlog, request latency, pool utilization and rejected work under load.

INTERVIEW QUESTION 101

How should async authorization differ from delayed business-job authorization?

EXPLAIN IN ONE BREATH

A queued job needs a defined approved-command and revalidation policy, not indefinite trust in a captured web authentication context.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
At submission -> authorize action, persist approved operation
At execution -> validate required current invariants or recorded approval
Audit -> record submitter and executing service
```

Why it works: Decide whether permission changes after submission should cancel the job or leave the prior approval valid. This is a business contract. Propagation wrappers solve context handling, not durable authorization semantics.

⚠ **Interview trap:** A user's short-lived token may expire before a queued task runs.

FOLLOW-UP QUESTION — ANSWERED

Which policy applies to a scheduled payment approved yesterday?

Reauthorize at execution using current policy or persist an approved, narrowly scoped business command with expiry and auditability.

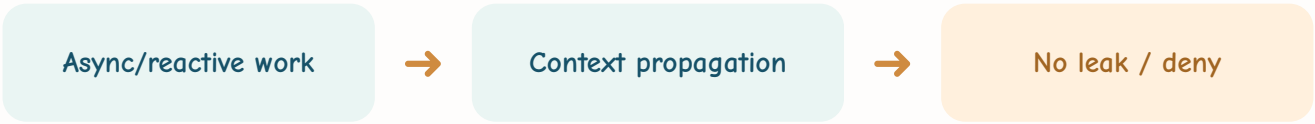
INTERVIEW QUESTION 102

Do virtual threads remove security-context lifecycle concerns?

EXPLAIN IN ONE BREATH

No. Cheap threads do not decide trusted identity, cleanup, credential lifetime, or cross-task propagation policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Virtual-thread request -> framework security integration
Custom task thread -> use supported context handling if required
Long-lived job -> explicit actor and approval policy
```

Why it works: Check the framework and executor integration rather than assuming every new thread inherits the correct context. Database, crypto and network capacity still constrain authentication throughput.

⚠ **Interview trap:** Changing thread type is not an authorization fix.

FOLLOW-UP QUESTION — ANSWERED

How would a context-leak regression test run across reused tasks?

Repeat tasks on reused executors, including exceptions and cancellation, asserting each sees only its approved identity and no leaked context.

18

Security testing and negative cases

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Test identity

Negative policy cases

No forbidden effect

103 · What does @WithMockUser test and what does it not test?

104 · How should MockMvc include Spring Security?

105 · How should CSRF be tested?

106 · How does the jwt() request helper differ from a real token test?

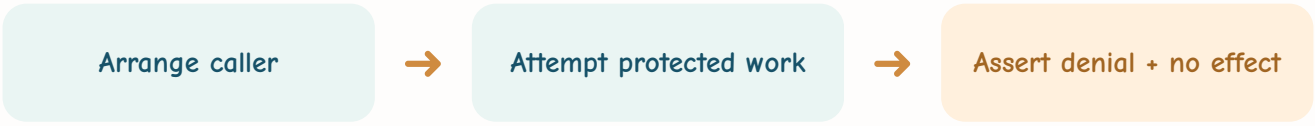
INTERVIEW QUESTION 103

What does @WithMockUser test and what does it not test?

EXPLAIN IN ONE BREATH

It supplies a test authentication for authorization behavior; it does not prove real password hashing, login or JWT signature verification.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@WithMockUser(authorities="orders:read")
@Test void allowedReader() { /* call protected boundary */ }
```

Why it works: Use it for policy tests and denied authority cases. Add actual authentication/decoder tests for credential trust. The tested call still needs the intended managed proxy or filter setup.

⚠ **Interview trap:** Mocked identity cannot prove audience rejection.

FOLLOW-UP QUESTION — ANSWERED

Which tests should use a signed JWT and real decoder?

@WithMockUser covers authorization under a supplied mock identity; use actual JWT signature/claim decoding tests separately.

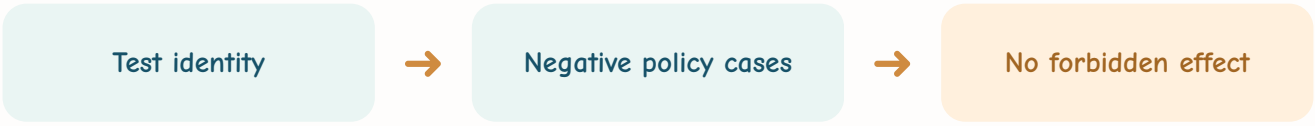
INTERVIEW QUESTION 104

How should MockMvc include Spring Security?

EXPLAIN IN ONE BREATH

Use the appropriate Boot slice/integration support or apply the security test integration so the real filter chain participates.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
mvc.perform(get("/orders/7").with(user("sai").authorities(
new SimpleGrantedAuthority("orders:read"))))
.andExpect(status().isOk());
```

Why it works: This assumes the handler fixture, matching policy and security filter integration are configured. Add anonymous and wrong-authority cases. A standalone controller test without security cannot validate route authorization.

⚠ **Interview trap:** addFilters=false removes important coverage.

FOLLOW-UP QUESTION — ANSWERED

How would a test prove which chain matched?

Test which SecurityFilterChain matched with route-specific permission behavior and Spring Security filter/debug instrumentation.

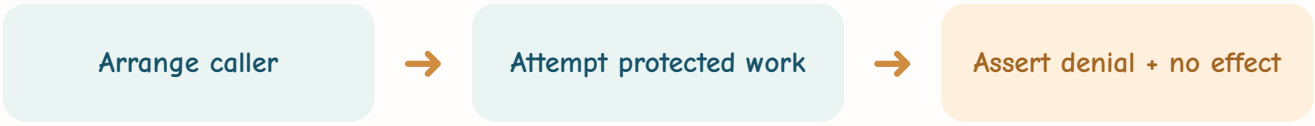
INTERVIEW QUESTION 105

How should CSRF be tested?

EXPLAIN IN ONE BREATH

Exercise missing, valid and invalid tokens on a real unsafe request under a CSRF-protected credential model.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
mvc.perform(post("/orders").with(user("sai")))
.andExpect(status().isForbidden());
mvc.perform(post("/orders").with(user("sai")).with(csrf()))
.andExpect(status().isCreated());
```

Why it works: The positive status assumes valid body, allowed permissions and a functioning handler. Add invalid-token helpers and login/logout lifecycle cases. Do not disable security to make a validation test pass unknowingly.

⚠ **Interview trap:** A GET success does not prove unsafe-method CSRF protection.

FOLLOW-UP QUESTION — ANSWERED

What would differ for a dedicated bearer-only chain?

Browser session POST needs CSRF; a reviewed Authorization-header-only chain can legitimately use a different CSRF policy.

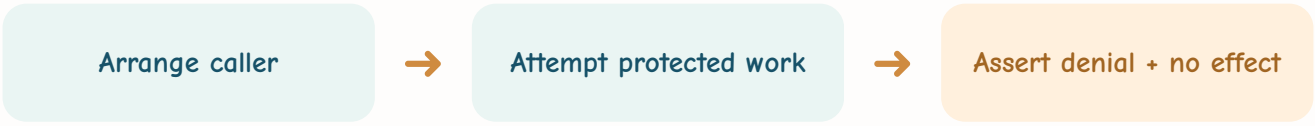
INTERVIEW QUESTION 106

How does the jwt() request helper differ from a real token test?

EXPLAIN IN ONE BREATH

It installs a test JWT-based authentication for authority policy; it does not perform real cryptographic token decoding.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
mvc.perform(get("/orders").with(jwt().authorities(
new SimpleGrantedAuthority("SCOPE_orders.read"))))
.andExpect(status().isOk());
```

Why it works: Use real signed tokens and the configured decoder for issuer, audience, expiry, key and algorithm cases. Both test levels are valuable because they isolate different failures. Avoid calling a fake claim test a signature verification test.

⚠ **Interview trap:** Mocking JwtDecoder can hide incorrect validators.

FOLLOW-UP QUESTION — ANSWERED

Which invalid tokens belong in an integration suite?

Real integration tests should reject altered signatures, unknown keys, wrong issuer/audience, expired/nbf and unsupported algorithms.

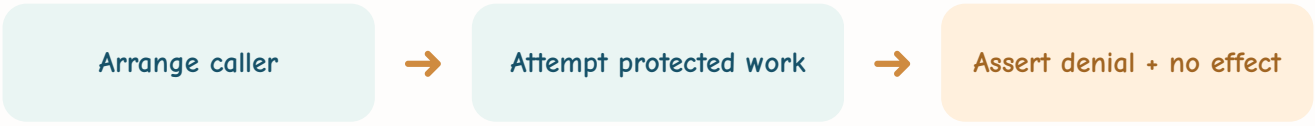
INTERVIEW QUESTION 107

How should method security be tested?

EXPLAIN IN ONE BREATH

Invoke the actual managed service boundary under allowed and denied identities, asserting both result and absence of denied side effects.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@WithMockUser(authorities="orders:read")
@Test void cannotApprove() {
assertThrows(AccessDeniedException.class, () -> service.approve(7));
verify(store, never()).approve(anyLong());
}
```

Why it works: The test must enable method security and use the proxied bean, not a manually constructed unadvised target. Also test ownership and alternate callers. The exact exception/result form can differ for reactive behavior.

⚠ **Interview trap:** A URL-only test misses non-HTTP service entry points.

FOLLOW-UP QUESTION — ANSWERED

How would self-invocation be exposed?

Call the managed proxied service from tests and separately demonstrate that an internal self-call bypasses proxy method advice.

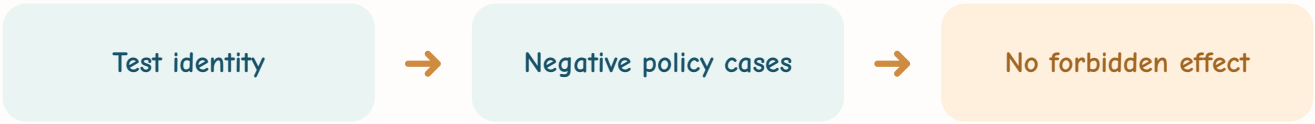
INTERVIEW QUESTION 108

What should a minimum negative security matrix include?

EXPLAIN IN ONE BREATH

Include absent/invalid credentials, wrong permission, wrong issuer/audience, expiry, tenant mismatch, unsafe browser requests, uncovered paths and credential leakage.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authentication failures -> no business side effect
Authorization failures  -> no forbidden state change
Success fixture -> exact intended permission and owner
```

Why it works: Test status and safe error contract, but also assert the invariant. A deny returning 403 after a database update is still a security defect. Add key rotation and identity-provider outage cases where relevant.

⚠ **Interview trap:** Happy-path coverage is not a security boundary proof.

FOLLOW-UP QUESTION — ANSWERED

How would you detect a token written to logs?

Emit synthetic requests with unique fake credentials and assert sanitized audit/log capture never contains the raw bearer token.

19

Operational security and abuse controls

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Security telemetry

Abuse / key event

Incident action

109 · How should security events be logged?

110 · How can adaptive password hashing become an availability concern?

111 · What should happen when JWKS or introspection is unavailable?

112 · How should credentials and client secrets be rotated?

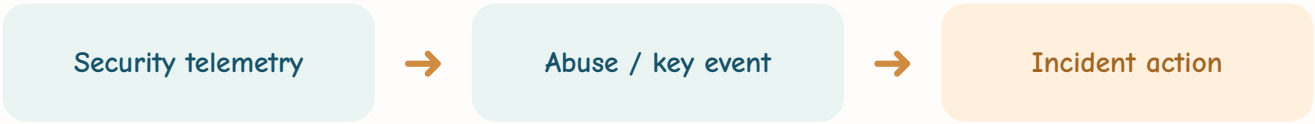
INTERVIEW QUESTION 109

How should security events be logged?

EXPLAIN IN ONE BREATH

Record safe actor/operation identifiers and failure categories with correlation, avoiding passwords, complete tokens and unnecessary sensitive claims.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Safe: category=invalid_audience, traceId=...
Unsafe: Authorization=Bearer <complete token>
Audit: actor, action, target, outcome, approved retention policy
```

Why it works: Diagnostic logging and durable business audit can have different consistency requirements. Restrict access and retention. Broad debug capture can expose headers through other infrastructure too.

⚠ **Interview trap:** Redacting one application logger does not sanitize ingress logs.

FOLLOW-UP QUESTION — ANSWERED

How would a sensitive action's audit be made durable?

Write an append-only or durable audit event with actor, action, object, outcome and correlation, without raw secrets.

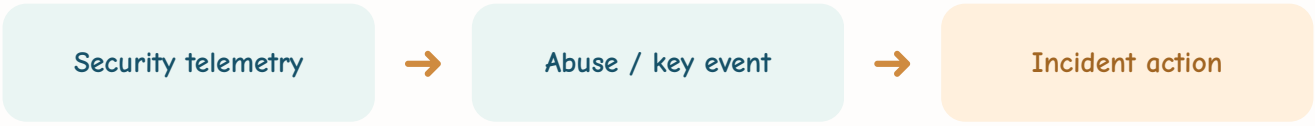
INTERVIEW QUESTION 110

How can adaptive password hashing become an availability concern?

EXPLAIN IN ONE BREATH

Each verification intentionally costs work, so unbounded login attempts can saturate CPU or memory without abuse controls.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Admission/rate policy -> bounded authentication work
Password verification -> measured adaptive cost
Failure monitoring -> account/IP/device signals with reviewed policy
```

Why it works: Tune using real hardware and traffic, not by weakening hashing blindly. Limit expensive work and design lockout to avoid letting attackers deny access to legitimate users easily. Multi-factor and recovery paths also need protection.

⚠ **Interview trap:** An unlimited executor can convert abuse into memory exhaustion.

FOLLOW-UP QUESTION — ANSWERED

Which signal distinguishes credential stuffing from an ordinary typo?

Use IP/account/device rate-and-failure patterns, credential reuse indicators and distributed sources to distinguish abuse from typos.

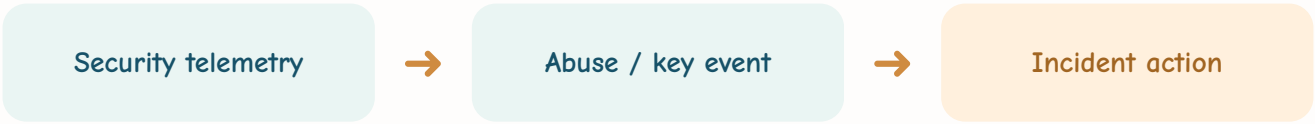
INTERVIEW QUESTION 111

What should happen when JWKS or introspection is unavailable?

EXPLAIN IN ONE BREATH

Follow the validation mode's supported cache and freshness policy; never authenticate an unvalidated token solely because the identity dependency is down.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Known cached key + valid applicable token -> supported local decision
Unknown key with failed refresh -> cannot establish signature trust
Introspection outage -> defined unavailable/failure contract
```

Why it works: Test behavior for existing versus new keys and cache expiry. Timeouts, circuit policy and metrics should show dependency failure without granting anonymous fallback privileges. Local validation has different coupling from per-request introspection.

⚠ **Interview trap:** Fail-open authentication is not a generic availability solution.

FOLLOW-UP QUESTION — ANSWERED

What operational alert would distinguish key rotation from an outage?

Alert on sudden unknown kid, JWKS refresh failures, validation error spikes and remote-fetch latency to separate rotation from outage.

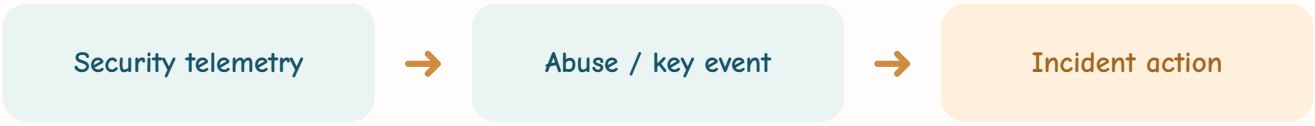
INTERVIEW QUESTION 112

How should credentials and client secrets be rotated?

EXPLAIN IN ONE BREATH

Use controlled secret/key versions, compatible overlap where appropriate, tested rollout, and explicit invalidation of compromised credentials.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Publish/activate replacement -> migrate clients
Verify successful use -> retire old version by policy
Compromise -> emergency revocation with reviewed blast radius
```

Why it works: Not every provider supports simultaneous secrets, so use its documented lifecycle. A configuration update must reach the actual token client or pool. Keep private signing keys and client secrets distinct.

⚠ **Interview trap:** Merely editing a secret file does not prove every running client adopted it.

FOLLOW-UP QUESTION — ANSWERED

How would you verify zero accidental fallback to old credentials?

Ensure old secret paths fail after the defined overlap; verify new credentials work and there is no implicit fallback to retired keys.

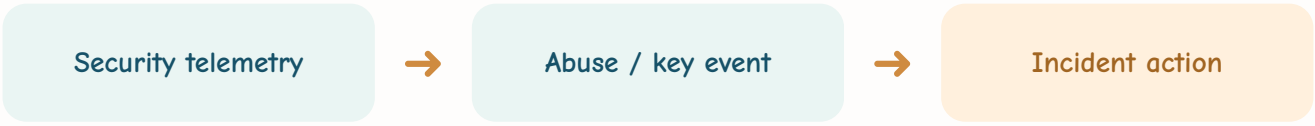
INTERVIEW QUESTION 113

How should actuator endpoints be secured in a security service?

EXPLAIN IN ONE BREATH

Expose only needed management capabilities and protect diagnostics with explicit chain, network and operator-access policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
/actuator/health -> limited probe policy
/actuator/prometheus -> monitoring identity/network
/actuator/env or heapdump -> sensitive diagnostic access
```

Why it works: Management exposure and authorization are different. A separate port is not automatically private. Test custom-chain behavior and avoid disclosing credentials or personally identifying claims.

⚠ **Interview trap:** PermitAll on /actuator/** can expose more than intended.

FOLLOW-UP QUESTION — ANSWERED

What must remain reachable for probes during an outage?

Keep liveness and narrowly authorized health probes reachable as designed; do not expose secret actuator endpoints publicly.

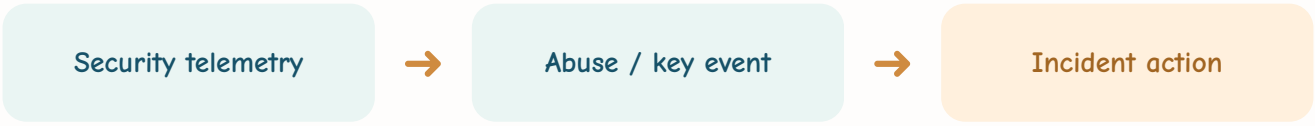
INTERVIEW QUESTION 114

How should revocation and permission changes be communicated operationally?

EXPLAIN IN ONE BREATH

Define required enforcement latency, identify which caches/sessions/tokens retain old rights, and monitor invalidation or expiry mechanisms.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Permission removed now
Session snapshot -> may retain old authorities
JWT -> may remain valid until expiry or revocation check
Policy -> refresh, invalidate, or revalidate as required
```

Why it works: The right choice depends on risk and availability. High-risk administrative actions may need fresher authority than ordinary cached reads. Audit policy changes and confirm the effective behavior.

⚠ **Interview trap:** Updating a database role does not instantly rewrite issued tokens.

FOLLOW-UP QUESTION — ANSWERED

How would a suspension test prove the required latency?

Suspend a test account and poll token/session-based protected actions until they fail; measure elapsed time against the revocation objective.

20

Modern API choices and legacy migration

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Old configuration → Review migration rules → Behavior tests

115 · Why should new configuration use SecurityFilterChain instead of WebSecurityConfigurerAdapter?

116 · What replaces old authorizeRequests and access-decision patterns in common new code?

117 · Why is matcher migration version-sensitive?

118 · What are the important Spring Security 6 context/CSRF migration distinctions?

INTERVIEW QUESTION 115

Why should new configuration use SecurityFilterChain instead of WebSecurityConfigurerAdapter?

EXPLAIN IN ONE BREATH

The adapter is removed in the modern baseline; bean-based chain configuration expresses supported composable policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@Bean
SecurityFilterChain security(HttpSecurity http) throws Exception {
// Supported lambda DSL and explicit authentication mechanism
return http.build();
}
```

Why it works: This empty sketch is not a complete protection policy. Migration must preserve chain scope, rule order, defaults and identity storage, not only change syntax. Use the actual baseline documentation.

⚠ **Interview trap:** A compiling migrated config can still leave routes uncovered.

FOLLOW-UP QUESTION — ANSWERED

How would old and new configurations be behavior-compared?

Exercise the same routes with anonymous, authorized and unauthorized callers against both implementations, including CSRF/session behavior.

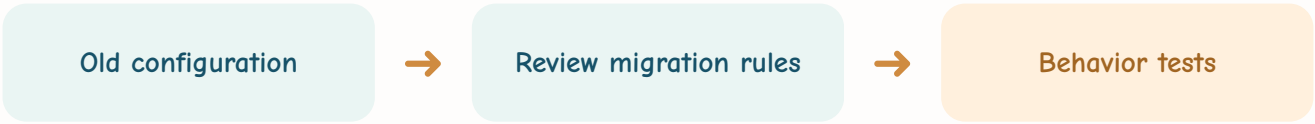
INTERVIEW QUESTION 116

What replaces old authorizeRequests and access-decision patterns in common new code?

EXPLAIN IN ONE BREATH

The modern request DSL uses authorizeHttpRequests and AuthorizationManager-based infrastructure, with migration appropriate to the policy's complexity.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
http.authorizeHttpRequests(a -> a
.requestMatchers("/admin/**").hasRole("ADMIN")
.anyRequest().authenticated());
```

Why it works: Legacy voters and metadata sources may need targeted translation rather than mechanical renaming. Method and reactive policies have their own supported APIs. Verify denied cases and custom decisions after migration.

⚠ **Interview trap:** An expression copied into the wrong DSL may not preserve semantics.

FOLLOW-UP QUESTION — ANSWERED

How would a custom authorization manager be tested?

Test AuthorizationManager rules for allowed and denied actor/action/object combinations, including missing/null authentication.

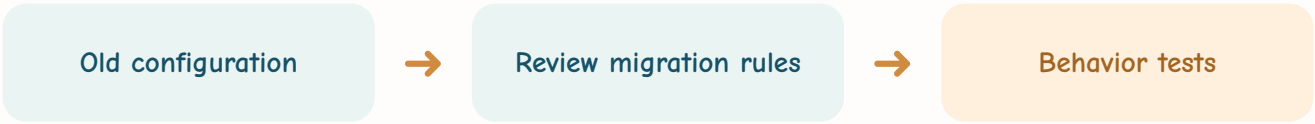
INTERVIEW QUESTION 117

Why is matcher migration version-sensitive?

EXPLAIN IN ONE BREATH

Matcher implementations and supported DSLs evolve, and path, servlet, dispatch and rule-selection semantics must be rechecked in the chosen release.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Old matcher configuration -> inventory paths and servlet assumptions
New supported requestMatchers -> test encoded and boundary paths
Fallback -> explicit coverage of unmatched traffic
```

Why it works: Do not assume an old antMatcher or a tutorial's package is available unchanged. Test trailing slash, context path, servlet path and alternate HTTP methods according to the app's routing. Avoid relaxing firewall checks to make odd paths work blindly.

⚠ **Interview trap:** Security matcher equivalence is a behavior question, not just compilation.

FOLLOW-UP QUESTION — ANSWERED

Which endpoint could become uncovered after migration?

After migrating matchers, assert coverage for unknown methods, forwarded routes, static paths and endpoints outside old prefixes.

INTERVIEW QUESTION 118

What are the important Spring Security 6 context/ CSRF migration distinctions?

EXPLAIN IN ONE BREATH

Modern context persistence uses explicit-save behavior for custom login paths, and CSRF client handling includes deferred/masked-token considerations.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Custom login -> invoke applicable session strategy and save context
SPA CSRF -> use supported token repository/handler integration
Test later request, not only the login response.
```

Why it works: Built-in mechanisms coordinate their supported behavior. A hand-written controller or filter needs the missing responsibilities. Compare against the old application's exact credential and session model.

⚠ **Interview trap:** Setting SecurityContextHolder alone can make login appear successful for one request only.

FOLLOW-UP QUESTION — ANSWERED

What must a custom persistent-login test assert?

Prove custom login saves to the selected SecurityContextRepository and that a second request is authenticated only when intended.

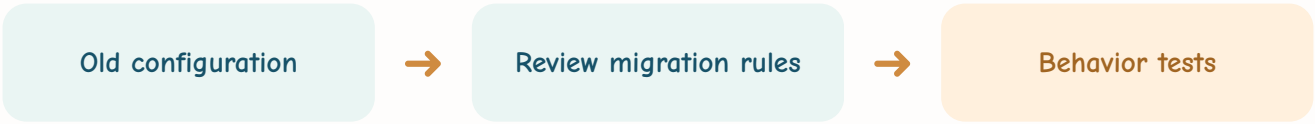
INTERVIEW QUESTION 119

How should Spring Security 7 be approached from a 6.x codebase?

EXPLAIN IN ONE BREATH

Use the selected migration guide and aligned Spring/Boot ecosystem, verifying APIs, matchers, serialization and behavior before switching production.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Inventory -> framework/security/authorization-server versions
Update supported DSL/types -> compile
Verify -> chain coverage, login, CSRF, method checks, token validation
```

Why it works: This guide keeps executable-style snippets on the Security 6.5/Boot 3.5 baseline. Security 7 details are release-specific and should not be inferred from the major number alone. Dependency alignment and third-party integration matter.

⚠ **Interview trap:** Overriding only the security version can create unsupported combinations.

FOLLOW-UP QUESTION — ANSWERED

Which test would detect changed session behavior?

Test login, next request, logout and fixation protections on the migrated stack; inspect actual session/cookie state.

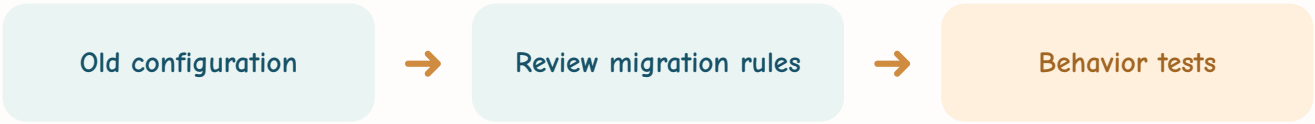
INTERVIEW QUESTION 120

How should a complete security design be reviewed?

EXPLAIN IN ONE BREATH

Start from assets, actors, trust boundaries and invariants, then verify identity, authorization, browser policy, token lifecycle, failure behavior and evidence.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Actor -> credential -> trusted identity
Action + object -> permission and ownership
Failure/rotation/logout -> defined effective behavior
Evidence -> negative tests, safe telemetry, reviewed operations
```

Why it works: Annotation count is not security assurance. Require a concrete policy for unknown outcomes and identity-service outages. Capture material tradeoffs and operational ownership.

⚠ **Interview trap:** A valid JWT and a passing 200 test do not prove isolation.

FOLLOW-UP QUESTION — ANSWERED

Which unresolved security requirement would block release?

Missing object-level access control, failing tenant isolation, permissive unknown routes or unvalidated JWT audience should block release.

21

Senior troubleshooting and architecture scenarios

Understand the trust boundary, describe the normal path and the denied path, and identify which test proves the rule.

CHAPTER MODEL

Incident evidence

Identify broken trust

Repair + regression

121 · An endpoint outside /api/** is unexpectedly public. What failed?

122 · A restrictive second chain never runs. Why?

123 · A logged-in user receives 403 on POST but GET works. What do you inspect?

124 · A token has a role claim but hasRole fails. Why?

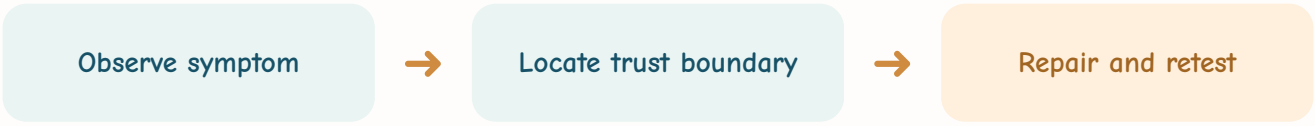
INTERVIEW QUESTION 121

An endpoint outside /api/** is unexpectedly public. What failed?

EXPLAIN IN ONE BREATH

The configured chain may cover only /api/** with no matching fallback, leaving other requests outside its protection.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Chain 1: securityMatcher("/api/**")
No other chain -> /reports may be uncovered
Repair -> deliberate catch-all chain and negative route tests
```

Why it works: Inspect actual chain match logs and the new route's path/dispatch behavior. Define a permitted or denied fallback rather than assuming anyRequest within one chain is global.

⚠ **Interview trap:** Adding a rule to an unselected chain has no effect.

FOLLOW-UP QUESTION — ANSWERED

How would a new endpoint be protected automatically?

A final lowest-priority deny-all chain catches new routes, and a regression test verifies unmapped URLs cannot reach handlers.

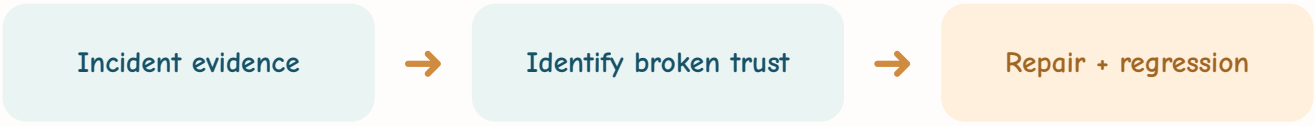
INTERVIEW QUESTION 122

A restrictive second chain never runs. Why?

EXPLAIN IN ONE BREATH

A higher-priority chain already matches, and chain selection uses the first match rather than merging all policies.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Order 1: all requests permitted
Order 2: /admin/** requires ADMIN
/admin/report -> first chain selected
```

Why it works: Narrow the first chain or reorder deliberately, then test representative paths across every chain. Also verify matcher rules inside the selected chain.

⚠ **Interview trap:** Two chains do not mean two cumulative security gates.

FOLLOW-UP QUESTION — ANSWERED

Where should a browser fallback sit relative to an API chain?

Put the narrow @Order(1) /api/** chain before the broader lower-priority browser catch-all chain.

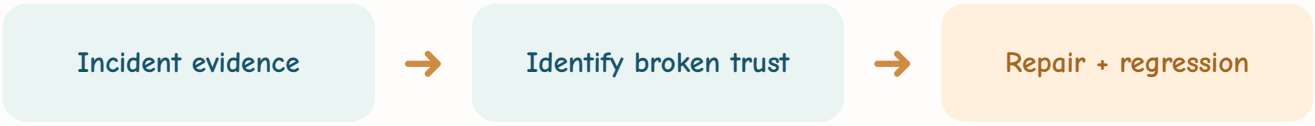
INTERVIEW QUESTION 123

A logged-in user receives 403 on POST but GET works. What do you inspect?

EXPLAIN IN ONE BREATH

Check CSRF, method-specific authority, matched chain and ownership before assuming authentication failed.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
GET /orders -> succeeds
POST /orders -> missing CSRF token in cookie app -> denied
Alternative -> POST requires a write authority absent from user
```

Why it works: Find the component responsible through safe diagnostics and reproduce with a focused test. Disabling CSRF globally can hide the issue while weakening the browser boundary.

⚠ **Interview trap:** 403 is not proof of one unique cause.

FOLLOW-UP QUESTION — ANSWERED

How would an invalid-token case differ?

Inspect CSRF rejection, role policy and request match; invalid credentials usually trigger authentication failure rather than a logged-in CSRF denial.

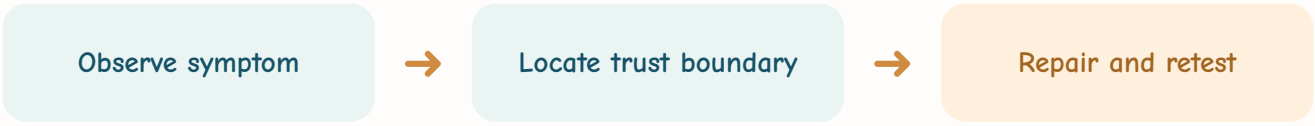
INTERVIEW QUESTION 124

A token has a role claim but hasRole fails. Why?

EXPLAIN IN ONE BREATH

The verified claim may not have been converted into the ROLE_ authority expected by the rule.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Claim: roles=["ADMIN"]
Current converter: only SCOPE_orders.read
Rule: hasRole("ADMIN") -> no ROLE_ADMIN -> deny
```

Why it works: Define a trusted converter and inspect the actual granted authorities without exposing credentials. Preserve scopes if both sets are needed. Never grant arbitrary roles just to make the test green.

⚠ **Interview trap:** A field named role has no automatic universal semantics.

FOLLOW-UP QUESTION — ANSWERED

How would converter tests cover malformed claim types?

Use typed claim validation and trusted allowlist conversion; test absent, wrong-type, extra and malicious role values.

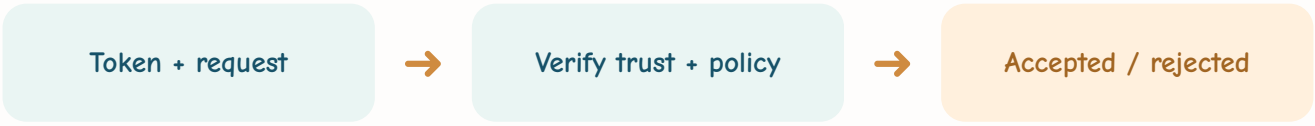
INTERVIEW QUESTION 125

A JWT is signed correctly but intended for another API. What is missing?

EXPLAIN IN ONE BREATH

The intended audience validation is absent or wrong, permitting a token outside this API's trust contract.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Token aud=profile-api
Request target=payments-api
Signature valid -> still reject for wrong audience
```

Why it works: Compose the audience check with issuer and time validation. Use actual negative signed-token tests, not only mocked JWT identities. The repair must cover every decoder/issuer path.

⚠ **Interview trap:** One issuer can legitimately issue tokens for many audiences.

FOLLOW-UP QUESTION — ANSWERED

What if aud contains multiple approved recipients?

Accept only configured target audiences; if multiple are present, require at least the resource's trusted audience and enforce policy.

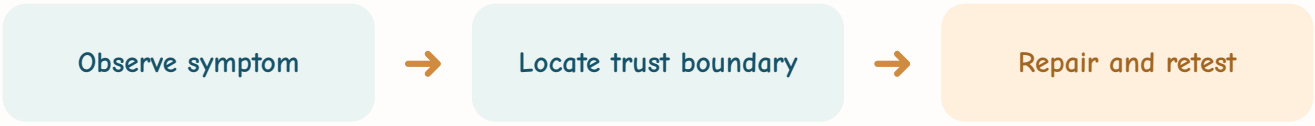
INTERVIEW QUESTION 126

A custom validator accepts expired tokens. What went wrong?

EXPLAIN IN ONE BREATH

It may have replaced default time/issuer validation instead of composing with it.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
decoder.setJwtValidator(audienceOnly); // Lost other checks
Repair: default issuer/time validator + audience validator
```

Why it works: Test expired, not-yet-valid, wrong-issuer and wrong-audience tokens independently. Check clock-skew policy and fixture times. A valid audience says nothing about expiry.

⚠ **Interview trap:** Passing one positive token cannot detect dropped validators.

FOLLOW-UP QUESTION — ANSWERED

Which library API makes composition explicit?

Use DelegatingOAuth2TokenValidator together with JwtValidators.createDefaultWithIssuer and a custom audience validator.

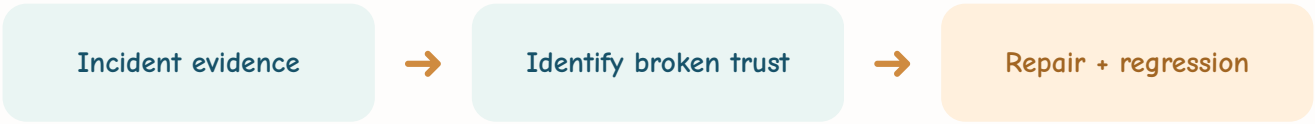
INTERVIEW QUESTION 127

Login works once but the next request is anonymous. Why?

EXPLAIN IN ONE BREATH

Custom authentication may have set the current context without saving it or applying the required session strategy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Login request -> SecurityContextHolder populated
Next request -> repository has no persisted context
Repair -> supported persistent login flow with explicit responsibilities
```

Why it works: Inspect selected session policy, repository, response cookie and browser transport. Built-in and custom flows differ. Test authentication on a second request with the returned session.

⚠ **Interview trap:** An API intended to be stateless should authenticate the next bearer request independently.

FOLLOW-UP QUESTION — ANSWERED

How would a cookie blocked by the browser change diagnosis?

Check whether the context was saved, response set cookie, browser sends it, session store survives restart, and cookie flags allow it.

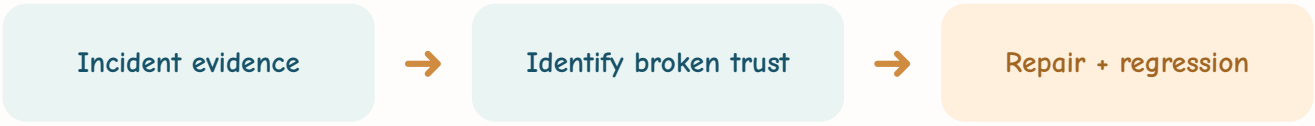
INTERVIEW QUESTION 128

Anonymous users appear authenticated in application code. Why?

EXPLAIN IN ONE BREATH

The code may treat a non-null or raw isAuthenticated Authentication as proof of end-user login despite anonymous tokens.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Authentication object exists -> not enough
AnonymousAuthenticationToken -> placeholder identity
Use supported trust-resolution/authorization semantics.
```

Why it works: Remove ad hoc checks and enforce policy at the proper boundary. Test anonymous and remember-me cases separately. A fabricated context must never bypass a business ownership check.

⚠ **Interview trap:** Anonymous authentication is not a verified account.

FOLLOW-UP QUESTION — ANSWERED

How would a service distinguish fresh from remembered authentication?

Use AuthenticationTrustResolver / framework request authorization to distinguish anonymous, remember-me and fully authenticated.

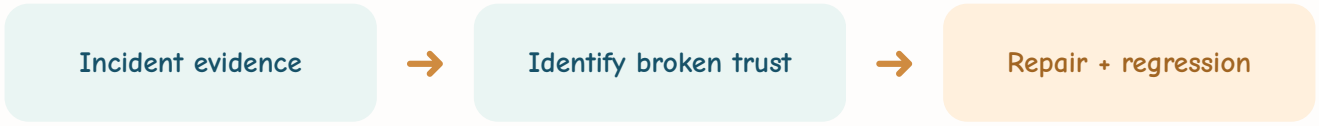
INTERVIEW QUESTION 129

A method annotation is ignored during an internal call. Why?

EXPLAIN IN ONE BREATH

Ordinary proxy-based method security is bypassed by direct self-invocation or an unmanaged target.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
this.approve(id) -> target self-call
CollaboratorProxy.approve(id) -> applicable advice boundary
```

Why it works: Refactor protected operations into a service boundary reached by every relevant caller. Verify method security is enabled and the method is eligible. Test a non-HTTP caller too.

⚠ **Interview trap:** Adding URL rules does not repair batch access automatically.

FOLLOW-UP QUESTION — ANSWERED

How would a plain new Service() behave?

A manually constructed non-managed Service lacks Spring proxy advice entirely; no @PreAuthorize interception happens.

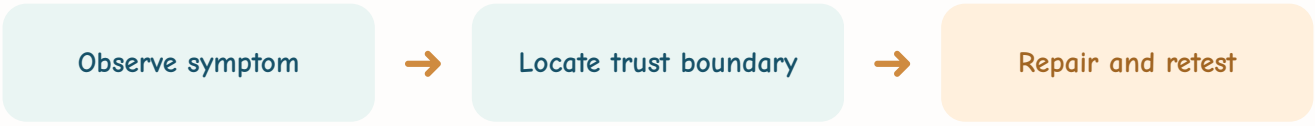
INTERVIEW QUESTION 130

A denied method changes the database before failing. What failed?

EXPLAIN IN ONE BREATH

Authorization may occur after invocation, or the check sits after the modifying work rather than protecting the operation before effect.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@PostAuthorize on approve -> target may already perform effect
Repair -> pre-authorization plus authoritative ownership/invariant
```

Why it works: Review transaction behavior separately; a later exception does not undo arbitrary HTTP side effects. Assert no forbidden state change in negative tests, not only an access-denied response.

⚠ **Interview trap:** A 403 response can coexist with a security defect if work already happened.

FOLLOW-UP QUESTION — ANSWERED

Which boundary must be atomic for ownership plus update?

Enforce ownership and state change atomically at the service/database write boundary; an after-the-fact check is too late.

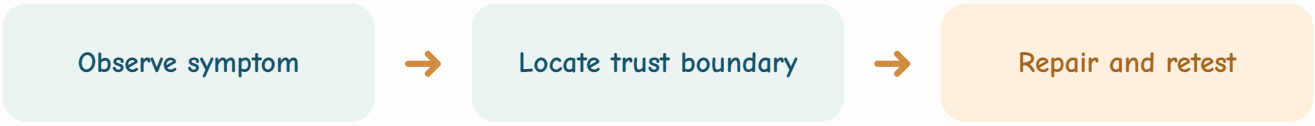
INTERVIEW QUESTION 131

A cache exposes another customer's order. What failed?

EXPLAIN IN ONE BREATH

The key/result omitted ownership dimensions or the only access check was skipped on a cache hit.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Outer: authorize trusted user and tenant every time
Inner: cache approved representation under correct key
Old unsafe entries: invalidate/rebuild deliberately
```

Why it works: A new key definition does not erase already stored entries. Check user-specific visibility, role changes and tenant isolation. Use immutable safe DTOs but remember immutability does not determine permission.

⚠ **Interview trap:** Valid login cannot correct wrong cached ownership.

FOLLOW-UP QUESTION — ANSWERED

How would a hit-path test expose the defect?

Warm the cache with A's data then request the same key under B; require access rejection or tenant-scoped cache key.

INTERVIEW QUESTION 132

A custom JWT filter runs twice. What do you inspect?

EXPLAIN IN ONE BREATH

The filter may be registered both with the servlet container and inside Spring Security, or dispatch handling may differ from expectations.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
@Component custom Filter -> Boot servlet registration
http.addFilterBefore(sameFilter, ...) -> security registration too
Review registration and actual request dispatches.
```

Why it works: Use a FilterRegistrationBean policy when necessary to avoid unintended container registration for a security-only filter. OncePerRequestFilter has specific async/error dispatch semantics; it is not a promise for every conceivable lifecycle.

⚠ **Interview trap:** Do not fix double execution by skipping authentication blindly.

FOLLOW-UP QUESTION — ANSWERED

Where should the filter's failure handler run?

Check registration as both servlet filter and Spring chain filter, REQUEST/ASYNC/ERROR dispatch, and OncePerRequestFilter behavior.

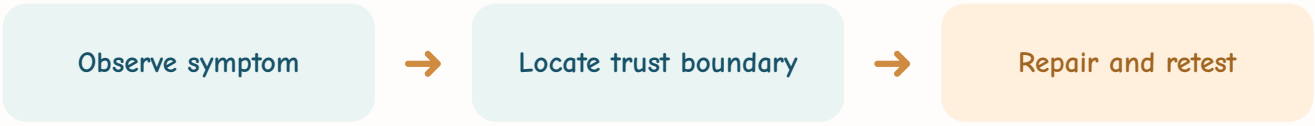
INTERVIEW QUESTION 133

Postman succeeds but browser preflight returns 401. What failed?

EXPLAIN IN ONE BREATH

CORS may not be processed before ordinary authentication in the selected chain or the expected origin/header policy is missing.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
OPTIONS with requested Authorization header -> no access token
CORS policy should decide preflight
Actual GET -> still requires valid bearer token
```

Why it works: Inspect both network requests and their returned headers. Enable the supported CORS integration and exact origin rules. Do not open all API methods to make OPTIONS pass.

⚠ **Interview trap:** A CORS error can conceal a separate actual-request failure.

FOLLOW-UP QUESTION — ANSWERED

Which test asserts allowed and denied origins?

Test allowed/disallowed Origin on preflight and actual requests; compare Access-Control headers and ensure policy does not bypass auth.

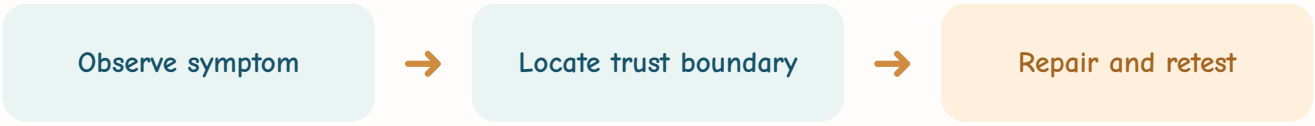
INTERVIEW QUESTION 134

JWT in a cookie is used after CSRF was disabled. What is wrong?

EXPLAIN IN ONE BREATH

The token format was mistaken for a credential-transport defense; browser-auto-sent token cookies can still support CSRF.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Cookie holds signed JWT
Browser attaches it to applicable requests
Unsafe request -> still needs deliberate CSRF protection
```

Why it works: Restore a reviewed browser policy or change to a genuinely bearer-header-only architecture with its own storage tradeoffs. Validate logout, origin and SameSite behavior. Token signature does not prove the victim intended the action.

⚠ **Interview trap:** Statelessness does not eliminate credential auto-submission.

FOLLOW-UP QUESTION — ANSWERED

What risks remain if JavaScript stores tokens instead?

JavaScript-readable token storage is exposed to XSS; use a storage/CSRF/CSP design that acknowledges this different risk.

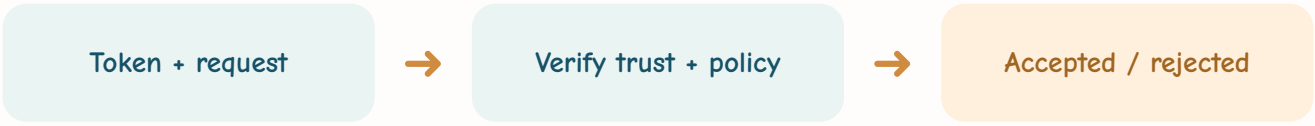
INTERVIEW QUESTION 135

A JWT still works after account suspension. Why?

EXPLAIN IN ONE BREATH

Local token validation does not automatically consult the changed account state or revoke its issued authority snapshot.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Suspend account -> database state changed
Issued JWT -> signature, issuer and exp still valid
Need defined revocation or fresh-policy check for required latency.
```

Why it works: Choose short lifetime, introspection, token-version or denylist policy based on risk and outage requirements. Include cached results and active sessions in the design. Verify actual enforcement delay.

⚠ **Interview trap:** Removing a role in a table does not edit an existing token.

FOLLOW-UP QUESTION — ANSWERED

Which operations need near-immediate revocation?

Financial transfers, admin actions, data export and irreversible operations often require near-immediate suspension enforcement.

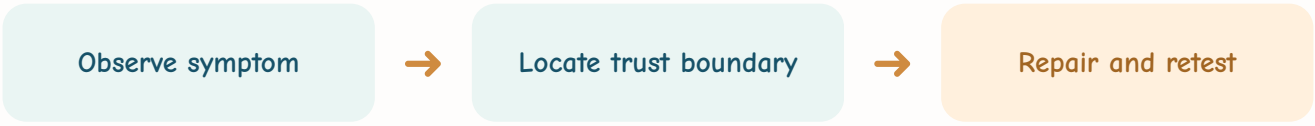
INTERVIEW QUESTION 136

Tokens fail after every authorization-server restart. What failed?

EXPLAIN IN ONE BREATH

Signing keys or issuer configuration may be regenerated or changed without the required durable key lifecycle and overlap.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Restart -> ephemeral signing key replaced
Old tokens -> published verification key no longer available
Repair -> protected persistent signing-key policy and rotation window
```

Why it works: Compare kid, issuer and JWKS across restarts, then validate a pre-restart token. Emergency key compromise differs from an ordinary restart. Never preserve a compromised key merely for availability.

⚠ **Interview trap:** Session restart and JWT signer restart are different failure modes.

FOLLOW-UP QUESTION — ANSWERED

How would planned retirement honor token lifetime?

Keep the old public key available for the maximum valid old-token lifetime plus clock skew; then remove it deliberately.

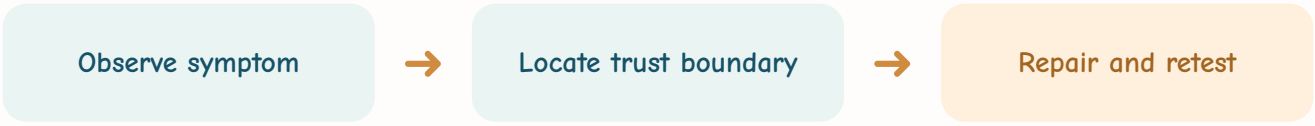
INTERVIEW QUESTION 137

A new kid fails only on some API instances. What do you inspect?

EXPLAIN IN ONE BREATH

Inspect trusted JWKS propagation, cache refresh, network access, clock/ claim policy and actual key material consistency.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Some pods cache old set -> new kid unresolved
Others refreshed -> verify succeeds
Observe refresh failure and overlap policy, not only token text.
```

Why it works: Plan publication before using a new signer and test cache behavior under failures. A bad kid or algorithm must still fail. Do not fetch an arbitrary header URL to bypass the problem.

⚠ **Interview trap:** Caching forever and disabling signature checks are unsafe repairs.

FOLLOW-UP QUESTION — ANSWERED

How would you retire old keys without a stampede?

Stagger refresh and cache compatible old/new verification keys; monitor kid misses without synchronized retry storms.

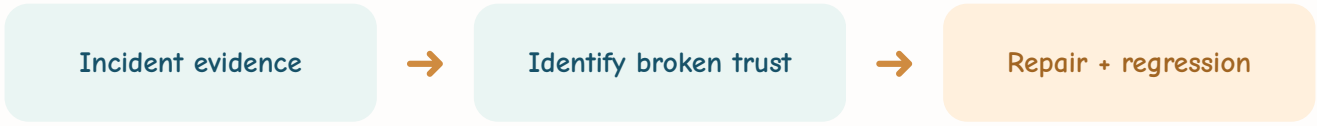
INTERVIEW QUESTION 138

An async task executes as the previous request's user. Why?

EXPLAIN IN ONE BREATH

Context was not cleaned up or was propagated using an unsuitable pooled-thread strategy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Task A sets user A -> no cleanup
Reused worker handles task B -> stale identity observed
Repair -> supported wrapper/restoration and explicit task actor policy
```

Why it works: Test at least two identities across reused workers and failure paths. Authentication state, MDC and request-owned data need separate proper cleanup. A durable task should not inherit accidental worker state.

⚠ **Interview trap:** InheritableThreadLocal does not solve task reuse.

FOLLOW-UP QUESTION — ANSWERED

How would context be restored after nested propagation?

Save previous context before delegation and restore it in finally after nested tasks; test exceptional exits on pooled workers.

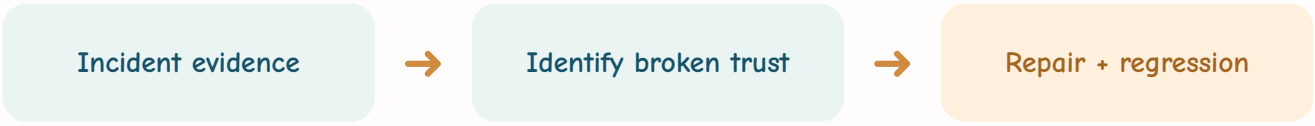
INTERVIEW QUESTION 139

A reactive service sees no authentication through SecurityContextHolder. Why?

EXPLAIN IN ONE BREATH

It is using imperative thread-local lookup instead of the reactive security context associated with the subscriber chain.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
ReactiveSecurityContextHolder.getContext()
.map(c -> c.getAuthentication().getName())
```

Why it works: Compose inside the supported publisher chain and test subscription context. Avoid separate subscribe calls that detach ownership. If bridging to imperative work, use a deliberate integration rather than an arbitrary current thread.

⚠ **Interview trap:** A debugger's thread identity is not a reactive authentication model.

FOLLOW-UP QUESTION — ANSWERED

How would WebTestClient supply test authentication?

Use mockJwt()/mockAuthentication() with WebTestClient's reactive security testing setup, then check actual reactive context.

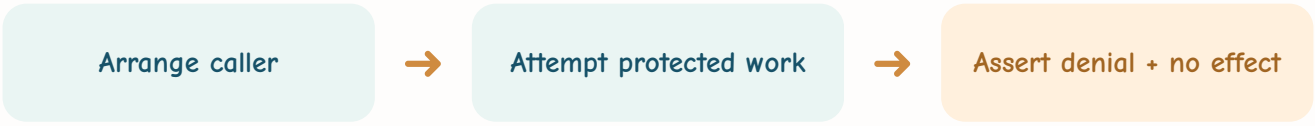
INTERVIEW QUESTION 140

Security tests pass while invalid signatures are accepted in production. What was missed?

EXPLAIN IN ONE BREATH

The suite may mock JwtDecoder or use jwt()/WithMockUser, testing authorization while bypassing real credential trust.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Policy test -> mock identity acceptable for rule checks
Decoder/integration test -> actual signed/invalid token required
Wrong key, algorithm, issuer, audience, time -> negative matrix
```

Why it works: Use a deterministic test issuer/key fixture and the actual validation configuration. Assert no business side effect for every rejected credential. A happy-path signed token still misses validator omissions.

⚠ **Interview trap:** More mocked claims do not prove signature verification.

FOLLOW-UP QUESTION — ANSWERED

Which production decoder bean must the integration test exercise?

Run a signed-token request against the same real JwtDecoder bean/configuration used in production, not a mocked jwt() shortcut.

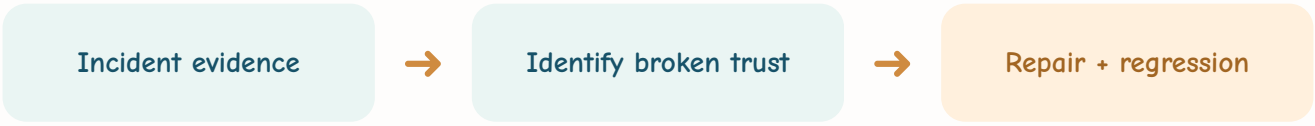
INTERVIEW QUESTION 141

An identity-provider outage results in permitAll fallback. Why is this dangerous?

EXPLAIN IN ONE BREATH

Availability handling is granting access without established identity or permission, violating the security boundary.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Introspection times out -> defined dependency failure
Do not convert failure into authenticated anonymous/default admin identity.
Known valid cached credentials -> only within reviewed freshness policy.
```

Why it works: Fix the fallback and assess exposure. Use bounds and observability to handle the outage without silently widening access. Public operations can have a separate intentional policy.

⚠ **Interview trap:** Circuit-breaker fallback must not invent privileges.

FOLLOW-UP QUESTION — ANSWERED

What degradation remains safe for the protected operation?

Allow only explicitly public, non-sensitive operations to degrade; protected reads/writes must fail closed when identity is unknown.

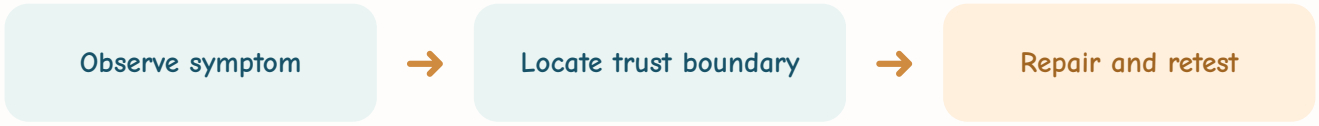
INTERVIEW QUESTION 142

A service accepts tokens from arbitrary iss URLs. What failed?

EXPLAIN IN ONE BREATH

Untrusted token data is choosing trust roots or network destinations instead of selecting from an approved issuer policy.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Incoming iss -> allowlist lookup only
Approved issuer -> predetermined trusted decoder policy
Unknown issuer -> reject without arbitrary discovery
```

Why it works: Besides overtrust, network fetches can create SSRF and resource exhaustion. Validate issuer again after choosing the policy. Keep audience and authority mapping specific to each approved issuer.

⚠ **Interview trap:** A valid signature from an attacker's issuer is not your trust contract.

FOLLOW-UP QUESTION — ANSWERED

How would adding a new tenant's issuer be reviewed?

Require a reviewed issuer allowlist, trusted metadata/JWKS origin and issuer-specific validation before onboarding a tenant.

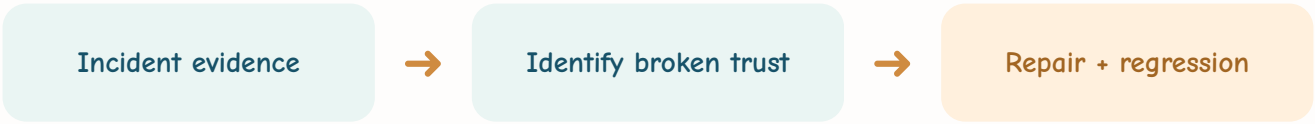
INTERVIEW QUESTION 143

A password-login surge exhausts CPU. How should it be handled?

EXPLAIN IN ONE BREATH

Bound expensive authentication work, detect abuse, and tune adaptive hashing with real evidence without weakening password protection indiscriminately.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Admission -> finite concurrency/rate budget
Verification -> measured cost
Monitoring -> failure categories, account/device signals, safe correlation
```

Why it works: Check legitimate peaks versus credential stuffing and avoid a lockout policy attackers can exploit to disable everyone. Protect recovery and MFA endpoints too. Load-test accepted and rejected paths.

⚠ **Interview trap:** Unlimited retries can amplify authentication pressure.

FOLLOW-UP QUESTION — ANSWERED

Which capacity limit should produce a controlled response?

Bound in-flight expensive password checks and emit 429/503 or equivalent controlled failure under overload; never bypass hashing.

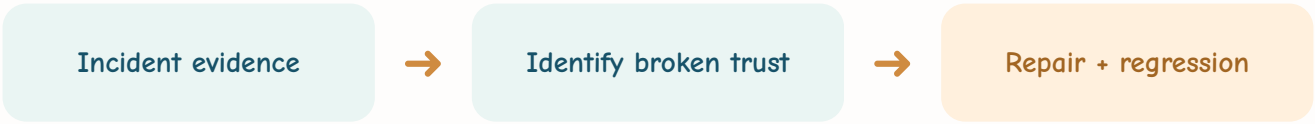
INTERVIEW QUESTION 144

How would you approve a security configuration for release?

EXPLAIN IN ONE BREATH

Require explicit chain coverage, trust roots, credential/browser policy, permissions and ownership, lifecycle/rotation rules, negative tests, and operational evidence.

QUICK VISUAL — WHAT HAPPENS?



ORIGINAL EXAMPLE / CODE

```
Review -> all routes and method entry points
Trust -> issuers, audiences, algorithms, keys, identity mappings
Proof -> denied cases produce no forbidden effect or credential leak
Operations -> logout, revocation, outage and recovery behavior
```

Why it works: Explain the concrete invariant and tradeoff for each important action. A compiling configuration is not proof of confidentiality, integrity or isolation. Name an owner for ongoing key/secret lifecycle and security incident handling.

⚠ **Interview trap:** Documentation that says 'JWT secured' is not enough evidence.

FOLLOW-UP QUESTION — ANSWERED

Which unanswered requirement would block approval?

Block approval for unknown authorization ownership, uncovered routes, unverified JWT audience/issuer, missing CSRF policy or untested denial.

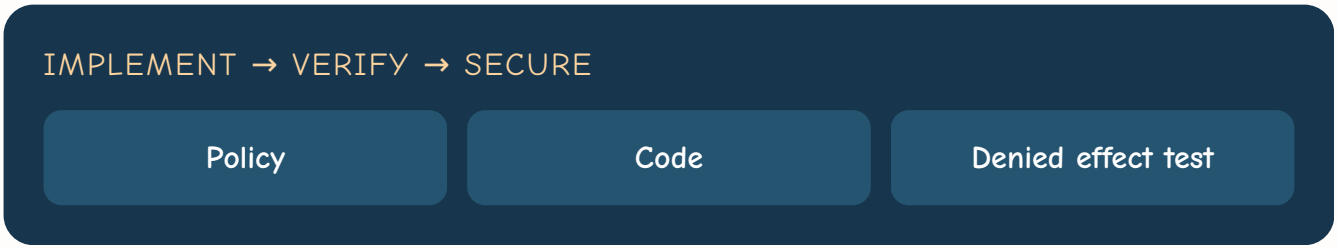
10

Hands-on Security Labs

The source guide's ten exercises, with their coding sketches and precise verification boundaries. Each needs a matching Spring Boot project and selected dependencies.

These excerpts are not a standalone runnable application. The source contains references to app-specific DTOs, controllers, test fixtures, identity stores and runtime config. Never treat a highlighted snippet alone as a production security policy.

Always test the negative path: missing or invalid token, wrong audience, wrong tenant, wrong method, missing CSRF, failed signing-key lookup and resource failure.



Source-code pack is included separately

HANDS-ON LAB C01

Build a browser session policy

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
@Bean
SecurityFilterChain browser(HttpSecurity http) throws Exception {
  http.authorizeHttpRequests(a -> a
      .requestMatchers("/public/**", "/login").permitAll()
      .requestMatchers("/admin/**").hasRole("ADMIN")
      .anyRequest().hasRole("USER"))
      .formLogin(Customizer.withDefaults())
      .logout(Customizer.withDefaults());
  return http.build();
}
```

VERIFICATION AND BOUNDARY POLICY

Configure a real user source and PasswordEncoder. Check anonymous redirect, USER success, USER denied admin, ADMIN policy as intended, missing/valid CSRF for POST, logout and a second request after login. An ADMIN does not automatically also have USER unless your authority policy supplies it; decide that mapping explicitly.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C02

Build a dedicated bearer-header API chain

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
@Bean @Order(1)
SecurityFilterChain api(HttpSecurity http) throws Exception {
  http.securityMatcher("/api/**")
      .csrf(c -> c.disable()) // Only this reviewed bearer-only boundary.
      .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(a -> a
          .requestMatchers(HttpMethod.GET, "/api/orders/**")
          .hasAuthority("SCOPE_orders.read")
          .anyRequest().denyAll())
      .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
  return http.build();
}
@Bean @Order(2)
SecurityFilterChain fallback(HttpSecurity http) throws Exception {
  http.authorizeHttpRequests(a -> a.anyRequest().denyAll());
  return http.build();
}
```

VERIFICATION AND BOUNDARY POLICY

The API must accept only explicit bearer headers, with no cookie/Basic authentication added to that boundary. Configure the trusted decoder from C03. Test allowed GET, wrong scope, invalid token, other HTTP methods and paths outside /api. If browser session endpoints are needed, replace the deny fallback with a reviewed browser chain rather than leaving paths uncovered.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C03

Compose issuer, time and audience checks

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
@Bean
JwtDecoder decoder(@Value("${security.issuer}") String issuer) {
  NimbusJwtDecoder d = JwtDecoders.fromIssuerLocation(issuer);
  OAuth2TokenValidator<Jwt> audience = jwt ->
      jwt.getAudience().contains("orders-api")
          ? OAuth2TokenValidatorResult.success()
          : OAuth2TokenValidatorResult.failure(
              new OAuth2Error("invalid_token", "Wrong audience", null));
  d.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
      JwtValidators.createDefaultWithIssuer(issuer), audience));
  return d;
}
```

VERIFICATION AND BOUNDARY POLICY

Supply security.issuer from a trusted setting, never directly from the incoming request. Test valid signature/claims, wrong signing key, wrong issuer, absent/wrong audience, expiry, not-before and unexpected algorithm. Discovery/key dependencies need deadlines and operational policy. Import the Spring Security OAuth2 core and jose JWT types appropriate to the baseline.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C04

Map scopes and approved application roles

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
JwtAuthenticationConverter converter() {
  JwtGrantedAuthoritiesConverter scopes = new JwtGrantedAuthoritiesConverter();
  JwtAuthenticationConverter out = new JwtAuthenticationConverter();
  out.setJwtGrantedAuthoritiesConverter(jwt -> {
    List<GrantedAuthority> granted = new ArrayList<>(scopes.convert(jwt));
    List<String> roles = jwt.getClaimAsStringList("roles");
    if (roles != null && roles.contains("ADMIN"))
      granted.add(new SimpleGrantedAuthority("ROLE_ADMIN"));
    return granted;
  });
  return out;
}
// Wire with jwt(j -> j.jwtAuthenticationConverter(converter())).
```

VERIFICATION AND BOUNDARY POLICY

This trusts ADMIN only if the issuer is authoritative for that application role and the client/grant policy permits it. Check scope preservation, absent roles, disallowed roles, malformed claim types, and every trusted issuer separately. Do not map an arbitrary client-controlled role into an administrator privilege.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C05

Verify CSRF before business work

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
@Test void deniesMissingCsrf() throws Exception {
  mvc.perform(post("/notes").with(user("sai").roles("USER")))
     .andExpect(status().isForbidden());
  verify(notes, never()).create(any());
}
@Test void permitsValidCsrf() throws Exception {
  mvc.perform(post("/notes").with(user("sai").roles("USER"))
      .with(csrf()).contentType(MediaType.APPLICATION_JSON)
      .content("{\"text\":\"hello\"}"))
     .andExpect(status().isCreated());
}
```

VERIFICATION AND BOUNDARY POLICY

Use MockMvc with the actual browser filter chain, mocked Notes service fixture, a real controller and its valid DTO. The positive status assumes the handler's defined contract. Add invalid tokens and login/logout token renewal. Static imports come from SecurityMockMvcRequestPostProcessors and MockMvc result matchers. A test using disabled filters does not verify this boundary.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C06

Configure a narrow CORS policy

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
@Bean
CorsConfigurationSource cors() {
  CorsConfiguration c = new CorsConfiguration();
  c.setAllowedOrigins(List.of("https://app.example"));
  c.setAllowedMethods(List.of("GET", "POST"));
  c.setAllowedHeaders(List.of("Authorization", "Content-Type"));
  c.setAllowCredentials(false); // Explicit bearer-header-only SPA policy.
  UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
  source.registerCorsConfiguration("/api/**", c);
  return source;
}
// Enable http.cors(Customizer.withDefaults()) in the matching chain.
```

VERIFICATION AND BOUNDARY POLICY

The chosen origin and credential mode are illustrative policy, not a universal recommendation. Check allowed/denied Origin, requested method/header combinations, OPTIONS without a bearer token and actual GET without credentials. An allowed preflight must not grant access to protected resources. Add cookie credentials only through a reviewed changed policy.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C07

Protect a reusable ownership boundary

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
@Configuration @EnableMethodSecurity
class MethodConfig {}
@Service
class OrderService {
  private final OrderStore store;
  OrderService(OrderStore store) { this.store = store; }
  @PreAuthorize("@orderPolicy.canRead(authentication, #p0)")
  public OrderView read(long id) { return store.read(id); }
}
```

VERIFICATION AND BOUNDARY POLICY

Implement OrderPolicy from verified identity and tenant memberships, with no client-header trust shortcut. Invoke the managed service proxy in tests. Check permitted owner, other owner, other tenant, anonymous caller and a message/batch entry path. Also test the query's tenant predicate. A policy that checks existence must avoid forbidden information disclosure under the API contract.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C08

Propagate and clear async context

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
Executor secured = new DelegatingSecurityContextExecutor(executor);
secured.execute(() -> protectedWork());
// Capture task identity under deliberate ownership.
// Wrapper restores/clears worker context on completion.
```

VERIFICATION AND BOUNDARY POLICY

Test user A, user B and an exceptional task on the same underlying worker. Assert every task sees only its assigned policy. Define what happens if permissions change before delayed execution. This is context propagation, not durable approval storage. Do not retain complete access tokens or browser session objects when an approved operation record is sufficient.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C09

Build real decoder negative tests

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
@Test void rejectsWrongAudience() {
  String token = fixture.signedAccessToken(
      issuer, "different-api", futureExpiry, trustedPrivateKey);
  assertThrows(JwtException.class, () -> decoder.decode(token));
}
@Test void rejectsWrongKey() {
  String token = fixture.signedAccessToken(
      issuer, "orders-api", futureExpiry, unrelatedPrivateKey);
  assertThrows(JwtException.class, () -> decoder.decode(token));
}
```

VERIFICATION AND BOUNDARY POLICY

The fixture is a helper you must implement with a vetted JWT library, deterministic test-only keys and configurable claims. Use the actual configured decoder, trusted key endpoint/fixture and stable clock policy. Add issuer, not-before, expiry and algorithm cases. jwt() and @WithMockUser intentionally do not replace these tests. No production private key belongs in the test repository.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

HANDS-ON LAB C10

Review and repair an unsafe configuration

SECURITY GOAL

Implement the exact policy shown in the source, then prove unauthorized requests cannot perform protected work.

WORKED SOURCE EXCERPT

```
Findings -> policy requirement -> minimal repair -> negative test
Uncovered path -> fallback chain -> deny unknown route
Audience missing -> composed validator -> wrong-audience rejection
Cookie credential -> CSRF policy -> unsafe request denied
Token logged -> safe event category -> log-redaction regression
```

VERIFICATION AND BOUNDARY POLICY

Produce a concrete review decision: each required invariant, its evidence, remaining risk and owner. Verify deployment proxy headers, management access, permission-change latency, key rotation and identity-provider outage. Do not approve merely because a valid token returns 200. Test that denied operations never perform the prohibited business effect.

Test status: Source-guide illustration; dependencies, fixtures and execution against Spring Security were not independently validated here.

Quick comparisons

Authentication = who you are	Authorization = what you can do
Role = grouped convention	Authority = exact permission
CSRF = forced browser action	CORS = browser origin policy
JWT = token representation	OAuth = delegated access
OIDC = identity on OAuth	JWT signature ≠ secrecy
401 = authentication needed	403 = denied operation

Strong senior answer: Name the actor, asset, trust boundary, exact policy, how the request can fail, and the negative test that proves no forbidden business change occurred.

Daily practice plan

Read the relevant section, explain the rule aloud, implement one small case and test at least one rejected request.

Day 01	Foundations + filter architecture	01-02
Day 02	Identity and providers	03
Day 03	Password hashing / user store	04
Day 04	Sessions and logout	05
Day 05	HTTP authorization	06
Day 06	Method ownership	07
Day 07	Weekly mock: chain tracing	01-07
Day 08	CSRF scenarios	08
Day 09	CORS + secure headers	09
Day 10	JWT structure and algorithms	10
Day 11	JWT decoder and scopes	11
Day 12	OAuth roles, PKCE, client grants	12
Day 13	OIDC login and federation	13
Day 14	Mock: token negatives	08-13
Day 15	Token lifetime and refresh	14

Daily practice plan

Read the relevant section, explain the rule aloud, implement one small case and test at least one rejected request.

Day 16	Authorization server + JWKS	15
Day 17	Multi-tenant API boundaries	16
Day 18	Reactive and async security	17
Day 19	Negative security test matrix	18
Day 20	Identity outage / abuse controls	19
Day 21	Migration compatibility	20
Day 22	Senior incident examples 1	21
Day 23	Senior incident examples 2	21
Day 24	Senior incident examples 3	21
Day 25	Build policy C01-C03	Labs
Day 26	Build policy C04-C06	Labs
Day 27	Ownership and async C07-C08	Labs
Day 28	Token negatives and audit C09-C10	Labs
Day 29	Full mock interviews	Mock
Day 30	Fix every unanswered negative case	Review

Verification notes

Preserved: All 144 question identifiers and exact question text, all 144 original follow-up prompts, 21 topic groupings and ten exercise source excerpts.

Expanded: Each follow-up now has a specific concise answer, plus color-coded steps, explanatory callouts and diagrams for quick revision.

Runtime status: This publishing transformation did not independently compile the Spring project, connect an identity provider or verify token behavior. The example snippets depend on actual Spring configuration, data access, keys and mocks.

Security: Tests should deny wrong subject/tenant, key, issuer, audience, origin, CSRF and expired tokens. An allowed request alone does not prove safe access control.

Commercial rights: Confirm permission to republish/adapt the source guide and any third-party material before selling it.

Primary documentation references from the source guide include the **Spring Security 6.5 Reference**, **Spring Boot 3.5 requirements**, and relevant OAuth/OIDC RFCs. See the source guide for the full primary-reference directory.